

Αναφορά πρωτη φασεις Project

Επεξεργασία και Διαχείριση Δεδομένων σε Δίκτυα Αισθητήρων

Αντωνιος Μουτσοπουλος

2021030024

Κοσμας Σπυριδακης

2021030059

Περιεχομενα

1. Αλλαγες στο κωδικα που μας δοθηκε
2. Σημαντικα κομματα κωδικα για καθε ερωτημα
3. Εξηγηση για την δομη των μηνυματων
4. Προσομοιώσεις
5. Δουλεια του καθε φοιτητη
6. Δουλεια για το δευτερο κομματι

1. Αλλαγες στο κωδικα που μας δοθηκε

Ο κωδικας που μας δοθηκε ειχε πολλα κομματα τα οποια δεν χρειαζομαστε για να υλοποιήσουμε ενα συστημα σαν τον TAG(Tiny AGgragation). Αυτα ηταν:

- Οτι εχει να κανει με Notify
- Οτι εχει να κανει με πιθανο χαμενο πακετο
- Και την αλλαγη πατερα ενος κομβου στο Routing
- Διαγραφει απο το RoutingMsg το senderID

Στο TAG ενας κομβος διαλεγει για πατερα αυτον που ακουσει πρωτον ενω ο κωδικας που μας δινotan εχει ενα IF ELSE οπου διάλεγε πρακτικα τον πατερα που ηταν πιο πανω στο δεντρο

Επισης δεν χρησιμοποιησαμε καθολου το Serial, και τα leds

2. Σημαντικα κομματα κωδικα για καθε ερωτημα

Ειχαμε να υλοποιήσουμε τρια ερωτηματα: Min, Sum , Avg

Αρχικα ας δουμε τι αλλαξαμε στο SimpleRoutingTree.h οπου ορίσαμε τα struct για αυτα τα 3 τυποι μηνυματων.

1. AggregationMin για τα μηνύματα του Min

```
28 typedef nx_struct AggregationMin {
29     nx_uint8_t minVal;    // minimum value (used for MIN)
30 } AggregationMin;
```

2. AggregationSum για τα μηνύματα του Sum

```
32 typedef nx_struct AggregationSum {
33     nx_uint16_t sum;      // sum value (used for SUM)
34 } AggregationSum;
```

3. AggregationAvg για τα μηνύματα του Avg

```
36 typedef nx_struct AggregationAvg {  
37     nx_uint16_t sum;           // sum value (used for AVG)  
38     nx_uint16_t count;        // count of values (used for AVG)  
39 } AggregationAvg;
```

Εγινε αλλαγή και στο struct του routing όπου προστεθηκε το aggType που είναι για το τι query θα εκτελεστεί

Επίσης ορίσαμε κάποιες σταθερές για να τις χρησιμοποιήσουμε

```
16     WINDOW_MILLI=80,  
17     EPOCH_PERIOD_MILLI=20000,  
18     AGGREGATION_TYPE_MIN=1,  
19     AGGREGATION_TYPE_SUM=2,  
20     AGGREGATION_TYPE_AVG=3,
```

Επειτα ας δούμε το wiring που καναμε στο SRTreeAppC.nc:

```
35 components new PacketQueueC(SENDER_QUEUE_SIZE) as AggMinSendQueueC;
36 components new PacketQueueC(RECEIVER_QUEUE_SIZE) as AggMinReceiveQueueC;
37 components new AMSenderC(AGGREGATION_TYPE_MIN) as AggMinSenderC;
38 components new AMReceiverC(AGGREGATION_TYPE_MIN) as AggMinReceiverC;
39
40 components new PacketQueueC(SENDER_QUEUE_SIZE) as AggAvgSendQueueC;
41 components new PacketQueueC(RECEIVER_QUEUE_SIZE) as AggAvgReceiveQueueC;
42 components new AMSenderC(AGGREGATION_TYPE_AVG) as AggAvgSenderC;
43 components new AMReceiverC(AGGREGATION_TYPE_AVG) as AggAvgReceiverC;
44
45 components new PacketQueueC(SENDER_QUEUE_SIZE) as AggSumSendQueueC;
46 components new PacketQueueC(RECEIVER_QUEUE_SIZE) as AggSumReceiveQueueC;
47 components new AMSenderC(AGGREGATION_TYPE_SUM) as AggSumSenderC;
48 components new AMReceiverC(AGGREGATION_TYPE_SUM) as AggSumReceiverC;
49
50 components new TimerMilliC() as EpochTimerC;
```

Εδώ έχουμε την Random και ένα Seed όπως και κάνουμε το wiring για τα 3 τυπoi μηνυμάτων που προσθεσαμε. Για κάθε πακετο χρειαζομαστε ένα Packet, AMPacket, AMSend, Receive και δυο PacketQueue ένα για να στελνουμε και ένα για να λαμβανουμε. Επίσης εδώ βλέπουμε και το Timer που βάλουμε για κάθε εποχη.



```
86     SRTreeC.Random->RandomC;
87     SRTreeC.AggMinPacket->AggMinSenderC.Packet;
88     SRTreeC.AggMinAMPacket->AggMinSenderC.AMPacket;
89     SRTreeC.AggMinAMSend->AggMinSenderC.AMSend;
90     SRTreeC.AggMinReceive->AggMinReceiverC.Receive;
91     SRTreeC.AggMinSendQueue->AggMinSendQueueC;
92     SRTreeC.AggMinReceiveQueue->AggMinReceiveQueueC;
93
94     SRTreeC.AggAvgPacket->AggAvgSenderC.Packet;
95     SRTreeC.AggAvgAMPacket->AggAvgSenderC.AMPacket;
96     SRTreeC.AggAvgAMSend->AggAvgSenderC.AMSend;
97     SRTreeC.AggAvgReceive->AggAvgReceiverC.Receive;
98     SRTreeC.AggAvgSendQueue->AggAvgSendQueueC;
99     SRTreeC.AggAvgReceiveQueue->AggAvgReceiveQueueC;
100
101     SRTreeC.AggSumPacket->AggSumSenderC.Packet;
102     SRTreeC.AggSumAMPacket->AggSumSenderC.AMPacket;
103     SRTreeC.AggSumAMSend->AggSumSenderC.AMSend;
104     SRTreeC.AggSumReceive->AggSumReceiverC.Receive;
105     SRTreeC.AggSumSendQueue->AggSumSendQueueC;
106     SRTreeC.AggSumReceiveQueue->AggSumReceiveQueueC;
107
108     SRTreeC.EpochTimer->EpochTimerC;
109
110     SRTreeC.Seed->RandomMlcgC.SeedInit;
```

Και επεिता ας δούμε και το σημαντικότερο κομμάτι. Της αλλαγής στο SRTreeC.nc.
Εδώ είχαμε τις πιο πολλές προσθήκες. Αρχικά εδώ βλέπουμε τα interface που κάνουμε use που είναι τα ίδια με αυτά που είδαμε και πριν στο wiring

```
45     uses interface Random as Random;
46     uses interface ParameterInit<uint16_t> as Seed;
47
48     uses interface PacketQueue as AggQuerySendQueue;
49     uses interface PacketQueue as AggQueryReceiveQueue;
50
51     uses interface Packet as AggMinPacket;
52     uses interface AMPacket as AggMinAMPacket;
53     uses interface AMSend as AggMinAMSend;
54     uses interface Receive as AggMinReceive;
55     uses interface PacketQueue as AggMinSendQueue;
56     uses interface PacketQueue as AggMinReceiveQueue;
57
58     uses interface Packet as AggSumPacket;
59     uses interface AMPacket as AggSumAMPacket;
60     uses interface AMSend as AggSumAMSend;
61     uses interface Receive as AggSumReceive;
62     uses interface PacketQueue as AggSumSendQueue;
63     uses interface PacketQueue as AggSumReceiveQueue;
64
65     uses interface Packet as AggAvgPacket;
66     uses interface AMPacket as AggAvgAMPacket;
67     uses interface AMSend as AggAvgAMSend;
68     uses interface Receive as AggAvgReceive;
69     uses interface PacketQueue as AggAvgSendQueue;
70     uses interface PacketQueue as AggAvgReceiveQueue;
71
72     uses interface Timer<TMilli> as EpochTimer;
```

Εδω βλέπουμε τις μεταβλητές που έχουμε ορίσει έχουμε τα 3 είδη μηνυμάτων το aggType για το τι query έχουμε και εκτελούμε. Το sample που είναι η μέτρηση του αισθητήρα ένα counter για την εποχή που βρισκόμαστε ένα agg_min, agg_sum, agg_count που αποθηκεύουν τις τιμές που λαμβάνουν από τα παιδιά και στέλνουν στον πατέρα 3 bool για το αν το ραδιο ετοιμάζεται να στείλει κάποιο μήνυμα συγκεκριμένου τύπου και μετά ορίζουμε τα tasks

```
101     message_t radioAggMinSendPkt;
102     message_t radioAggAvgSendPkt;
103     message_t radioAggSumSendPkt;
104     uint8_t aggType;
105     uint8_t sample;
106     uint16_t epochCounter;
107     uint8_t agg_min;
108     uint16_t agg_sum;
109     uint8_t agg_count;
110     bool AvgSendBusy=FALSE;
111     bool MinSendBusy=FALSE;
112     bool SumSendBusy=FALSE;
113     //END ADDED
114
115     task void sendRoutingTask();
116     task void receiveRoutingTask();
117     task void sendAggMinTask();
118     task void receiveAggMinTask();
119     task void sendAggSumTask();
120     task void receiveAggSumTask();
121     task void sendAggAvgTask();
122     task void receiveAggAvgTask();
```

Και επείτα έχουμε 3 συναρτήσεις για να μπορούμε να αλλάζουμε τα bool



```
126     void setAvgSendBusy(bool state){
127         atomic{
128             AvgSendBusy = state;
129         }
130     }
131
132     void setMinSendBusy(bool state){
133         atomic{
134             MinSendBusy = state;
135         }
136     }
137
138     void setSumSendBusy(bool state){
139         atomic{
140             SumSendBusy = state;
141         }
142     }
```


Και τώρα παμε στο κομμάτι που εκτελείται όταν bootάρει ο αισθητήρας.

Αρχικά ανοίγει το ραδιο καλώντας το `RadioControl.start` που αυτό όταν εκτελεστεί και τελειώσει θα τρέξει ο κωδικός στο `RadioControl.startDone` που θα δούμε πιο κάτω και φτιαχνουμε ένα `Seed` με βάση το ρολοι και το ID του κομβού, αυτό το κάνουμε για να διαφέρουν τα random αναμεταξί τον αισθητήρων. Έπειτα αρχικοποιούμε τα state σε false και αρχικοποιούμε όλα τα agg το roundcounter και το sample σε 0 εκτός το agg_min όπου θέλουμε να του βάλουμε την μεγαλύτερη δυνατή τιμή ώστε με ότι και να το συγκρίνουμε να βγαίνει μεγαλύτερο και να αλλάζει τιμή.

Μετά έχουμε έναν έλεγχο για να τρέξουμε διαφορετικό κομμάτι αν είναι ο πρώτος κόμβος που θα είναι ο Root και διαφορετικό αν είναι κάποιος άλλος κόμβος.

Αν είμαστε ο root θέλουμε να αρχικοποιήσουμε το βάθος μας στο δέντρο σε 0 αφού είμαστε ο root και να βάλουμε σαν πατέρα το 0, αν και δεν παίζει ρολο γιατί μετεπειτα θα δούμε ότι ο root δεν χρησιμοποιεί τον πατέρα αφού δεν στέλνει μήνυμα προς τα πάνω.

Αν τώρα είμαστε κάποιος άλλος κόμβος αρχικοποιούμε αυτές τις δύο μεταβλητές σε -1 για να δίδουμε ότι ακόμα δεν έχουν πάρει τιμές

```
235 event void Boot.booted(){
236     /////// αρχικοποιήσι radio και serial
237     call RadioControl.start();
238     call Seed.init(clock()+TOS_NODE_ID);
239
240     setRoutingSendBusy(FALSE);
241     setAvgSendBusy(FALSE);
242     setMinSendBusy(FALSE);
243     setSumSendBusy(FALSE);
244     //setNotifySendBusy(FALSE); no tag
245     #ifdef SERIAL_EN
246         setSerialBusy(FALSE);
247     #endif
248     roundCounter =0;
249     agg_count=0;
250     agg_sum=0;
251     agg_min=0xFFFF;
252     sample=0;
253     if(TOS_NODE_ID==0){
254         #ifdef SERIAL_EN
255             call SerialControl.start();
256         #endif
257         curdepth=0;
258         parentID=0;
259         epochCounter=0;
260         dbg("Boot", "curdepth = %d , parentID= %d \n", curdepth , parentID);
261         #ifdef PRINTFDBG_MODE
262             printf("Booted NodeID= %d : curdepth= %d , parentID= %d \n", TOS_NODE_ID ,curdepth , parentID);
263             printf fflush();
264         #endif
265     }
266     else
267     {
268         curdepth=-1;
269         parentID=-1;
270         dbg("Boot", "curdepth = %d , parentID= %d \n", curdepth , parentID);
271         #ifdef PRINTFDBG_MODE
272             printf("Booted NodeID= %d : curdepth= %d , parentID= %d \n", TOS_NODE_ID ,curdepth , parentID);
273             printf fflush();
274         #endif
275     }
276 }
```

Ας δούμε και το `RadioControl.startDone` που αναφεραμε πιο πάνω

```

278 event void RadioControl.startDone(error_t err){
279     if (err == SUCCESS)
280     {
281         dbg("Radio" , "Radio initialized successfully!!!\n");
282         #ifdef PRINTFDBG_MODE
283         printf("Radio initialized successfully!!!\n");
284         printf fflush();
285         #endif
286
287         //call RoutingMsgTimer.startOneShot(TIMER_PERIOD_MILLI);
288         //call RoutingMsgTimer.startPeriodic(TIMER_PERIOD_MILLI);
289         //call LostTaskTimer.startPeriodic(SEND_CHECK_MILLIS);
290         if (TOS_NODE_ID==0)
291         {
292             call RoutingMsgTimer.startOneShot(TIMER_FAST_PERIOD);
293
294         }
295     }
296     else
297     {
298         dbg("Radio" , "Radio initialization failed! Retrying...\n");
299         #ifdef PRINTFDBG_MODE
300         printf("Radio initialization failed! Retrying...\n");
301         printf fflush();
302         #endif
303         call RadioControl.start();
304     }
305 }

```

Εδώ το σημαντικό κομμάτι είναι ότι αν για κάποιο λόγο δεν καταφέρει να ανοίξει το ραδιό ξανά προσπαθεί και αν ανοίξει με επιτυχία και είμαστε ο root αρχίζει ένα timer για να στείλει το routing μήνυμα.

Για το routing μήνυμα δεν θα βάλουμε εικόνα γιατί ο κώδικας έχει μείνει σχεδόν ίδιος με αυτόν που μας δόθηκε αλλά θα αναφέρουμε τις αλλαγές: προστέθηκε στην γραμμή 388 μια γραμμή που εκτελείται μόνο αν είμαστε ο root για να διαλέξει τυχαία τι τύπου query θα εκτελέσουμε και στην γραμμή 420 βάζουμε αυτήν την τιμή στο πακέτο που θα στείλουμε. Επίσης επειδή όλοι οι κομβοί θα στείλουν αυτό το routing μήνυμα βάλουμε να αρχίζει το Timer τους για τις εποχές με διαφορετική αρχή ο καθένας αναλογα το βάθος τους στο δέντρο. Αλλά ακόμα και στο ίδιο βάθος να είναι έχουμε βάλει ένα random offset για να μειώσουμε την πιθανότητα συγκρούσεων αυτό το βλέπουμε στην γραμμή 689.

```

689 uint8_t random_offset = call Random.rand16() % 10;
690 dbg("Epoch","Start epoch timer for node %d \n", TOS_NODE_ID);
691 call EpochTimer.startPeriodicAt(-(curdepth*WINDOW_MILLI)+random_offset,EPOCH_PERIOD_MILLI);

```

Η τιμή του offset περνούν τις τιμές από το 0 – 9 πρακτικά είναι σαν να δίνουμε στον κάθε κόμβο ένα μέγεθος παραθύρου για να ακουει γύρω στα 10-15 msec ενώ το συνολικό παράθυρο για κάθε εποχή που έχουμε δώσει σε κάθε κόμβο για κάθε εποχή όπως είδαμε πιο πριν είναι 80 msec αρα 10-15 για αποστολή 55 για επεξεργασία και 10 για αποστολή. Αν και δεν ανοιγοκλείνουμε το ασύρματο. Το μόνο που κάνουμε είναι να αρχίζουμε να επεξεργαζόμαστε τα δεδομένα μόλις βαρεσει το timer και να τα στέλνουμε μόλις τελειώσει η επεξεργασία. Αλλά θεωρητικά θα μπορούσαμε να πούμε ότι έχουμε κάτι τέτοιο σαν χρονούς παράθυρον.

Τώρα ας δούμε τι ακριβώς κάνουμε κάθε φορά που βαραιο το χρονόμετρο της εποχής

```
1132 epochCounter+=1;
1133 if(epochCounter==1){
1134     sample = (call Random.rand16() % 60) + 1; // random sample between 1 and 60
1135 }else{
1136     sample = (sample * ((call Random.rand16() % 40) + 80)) / 100; // * 0.8 to 1.2
1137     if(sample > 60){
1138         sample = 60;
1139     }else if(sample < 1){
1140         sample = 1;
1141     }
1142 }
```

Εδώ βλέπουμε ότι κάθε φορά που βαραιο το χρονόμετρο αυξανουμε την εποχή κατά ένα, και περνουμε την μετρησή μας. Αν είναι η πρώτη εποχή θα βγάλουμε έναν τυχαίο αριθμό από το 1 - 60 ενώ αν είναι κάποια άλλη εποχή θα βγάλουμε ένα τυχαίο ποσοστό από το 80% - 120% και θα το πολλαπλασιάσουμε με την προηγούμενη μετρησή ώστε να έχουμε μέγιστη διαφορά από την προηγούμενη μετρησή $\pm 20\%$ και έχουμε και δύο ελέγχους ώστε να μην ξεφευγει από το 1-60.

Επειτα έχουμε ένα μεγάλο IF για να δούμε τι query εκτελούμε. Ας δούμε πρώτα το Min

Η πρώτη γραμμή είναι ένας έλεγχος για το αν το sample είναι μικρότερο από αυτά που έχουμε λαβεί από τα παιδιά μας αν είναι μικρότερο το βάζουμε σε ένα τεμπ αλλιώς αν το agg_min είναι μικρότερο γράφει αυτό σε ένα τεμπ.

Μετά πάμε σε ένα IF για να δούμε αν είμαστε στο root ή όχι αν είμαστε στο root απλά τυπώνει το αποτέλεσμα αλλιώς αν είμαστε σε κάποιον άλλον κομβό αρχίζουμε να ετοιμάζουμε τα πεδία του μηνύματος για να το στείλουμε το βάζουμε στην ουρά και καλούμε το task sendAggMinTask για να στείλουμε το μήνυμα

```
1145 dbg("Sample","NodeID= %d : AggregationMin sample= %u , agg_min= %u \n", TOS_NODE_ID, sample, agg_min);
1146 if(sample < agg_min){
1147     temp = sample;
1148     agg_min = sample;
1149 }else{
1150     temp = agg_min;
1151 }
1152
1153 if(TOS_NODE_ID==0){
1154     dbg("Results","***\n");
1155     dbg("Results","AGG RESULT epoch=%u MIN=%u \n", epochCounter, agg_min);
1156     dbg("Results","***\n");
1157 }else{
1158     AggregationMin* msgMin;
1159     if(call AggMinSendQueue.full()){
1160         dbg("Min","AggMinSendQueue is FULL!!! \n");
1161         return;
1162     }
1163     msgMin = (AggregationMin*) (call AggMinPacket.getPayload(&temp, sizeof(AggregationMin)));
1164     if(msgMin==NULL){
1165         dbg("Min","EpochTimer.fired(): No valid payload... \n");
1166         return;
1167     }
1168     atomic{
1169         msgMin->minVal = agg_min;
1170         //msgMin->epoch = epochCounter;
1171     }
1172     dbg("Min","NodeID= %d : AggregationMin value= %u \n", TOS_NODE_ID, agg_min);
1173     call AggMinAMPacket.setDestination(&temp, parentID);
1174     call AggMinPacket.setPayloadLength(&temp, sizeof(AggregationMin));
1175     enqueueDone = call AggMinSendQueue.enqueue(temp);
1176
1177     if(enqueueDone==SUCCESS){
1178         if(call AggMinSendQueue.size()==1){
1179             dbg("Min","SendAggMinTask() posted!!\n");
1180             post sendAggMinTask();
1181         }
1182         dbg("Min","AggregationMin enqueued successfully in SendingQueue!!\n");
1183     }
1184 }
```

Εδώ βλέπουμε το task που είναι υπεύθυνο για την αποστολή του Min αρχικά έχουμε κάποιους ελέγχους για να δούμε ότι όλα έχουν πάει καλά και ότι δεν έχουμε αρχίσει να στέλνουμε άλλο πακέτο που αφορά min μετά βγαζουμε από την ουρά το πακέτο μας βλέπουμε αν έχει το σωστό μέγεθος και το στέλνουμε. Όταν σταλθεί μπενούμε στο AggMinAMSend.sendDone όπου γυρίζουμε το state ξανά σε false και βλέπουμε αν υπάρχει κάποιο άλλο μήνυμα που πρέπει να σταλεί.

```
1266 task void sendAggMinTask(){
1267     uint8_t mlen;
1268     uint16_t mdest;
1269     error_t sendDone;
1270
1271     dbg("Min","SendAggMinTask(): Starting...\n");
1272     if(call AggMinSendQueue.empty()){
1273         dbg("Min","SendAggMinTask(): Q is empty!\n");
1274         return;
1275     }
1276     if(MinSendBusy){
1277         dbg("Min","SendAggMinTask(): MinSendBusy= TRUE!!!\n");
1278         post sendAggMinTask();
1279         return;
1280     }
1281     radioAggMinSendPkt = call AggMinSendQueue.dequeue();
1282     mlen = call AggMinPacket.payloadLength(&radioAggMinSendPkt);
1283     mdest = call AggMinAMPacket.destination(&radioAggMinSendPkt);
1284     if(mlen!=sizeof(AggregationMin)){
1285         dbg("Min","\t\t SendAggMinTask(): Unknown message!!!\n");
1286         return;
1287     }
1288
1289     sendDone = call AggMinAMSend.send(mdest, &radioAggMinSendPkt, mlen);
1290     if(sendDone == SUCCESS ){
1291         dbg("Min","SendAggMinTask(): Send returned success!!!\n");
1292         setMinSendBusy(TRUE);
1293     }else{
1294         dbg("Min","send failed!!!\n");
1295     }
1296 }
```

Με τον ίδιο τρόπο στέλνουμε όλα τα πακέτα είτε είναι Min είτε Sum είτε Avg απλά αλλάζουν ο Sender και ότι άλλο είναι για να είναι του αντιστοιχού τυπού. Ας γυρίσουμε πίσω στο χρονομετρο της εποχής να δούμε μέσα στο μεγάλο αυτό IF τι γίνεται αν το query μας είναι Sum.

Αρχίζουμε αρχικά και προσθετούμε την μετρηση του αισθητήρα στο αθροισμα που έχει λαβει απο τα παιδια και μετα οπως και στο Min ειχαμε εναν ελεγχο για αν ειμαστε root ετσι και εδω αν ειναι root τυπώνει το αποτελεσμα αλλιως αρχιζει και ετοιμαζει το μηνυμα που θα στείλει. Με τον ιδιο τροπο ακριβως ειναι και το Avg απλα στην αρχη αυξανει το counter κατα ενα και προσθετη το sample στο agg_sum.

[illegible]

Και με αυτά τα λίγα φτάνουμε στο σημείο που εστώ όλα έχουν πάει καλά πως ο αισθητήρας λαμβάνει τα μηνύματα. Επειδή είναι πολύ παρομοια για όλα θα γράψουμε στην αναφορά μόνο το ένα με φωτογραφία και θα πούμε απλά τι διαφέρει στα άλλα

Όταν γίνεται receive ένα πακέτο καλείται το .receive το αναλογου πακέτου εστω τωρα για το min θα καλεστεί το event AggMinReceive.receive

```
1387 event message_t* AggMinReceive.receive(message_t* msg, void* payload, uint8_t len){
1388     error_t enqueueDone;
1389     message_t tmp;
1390     uint16_t msource;
1391
1392     msource = call AggMinAMPacket.source(msg);
1393     dbg("Min", "### AggMinReceive.receive() start ##### \n");
1394     dbg("Min", "Something received!!! from %u\n", msource);
1395
1396     atomic{
1397         memcpy(&tmp, msg, sizeof(message_t));
1398     }
1399     enqueueDone = call AggMinReceiveQueue.enqueue(tmp);
1400     if(enqueueDone == SUCCESS){
1401         dbg("Min", "posting receiveAggMinTask()!!!! \n");
1402         post receiveAggMinTask();
1403     }else{
1404         dbg("Min", "AggMinMsg enqueue failed!!! \n");
1405     }
1406     return msg;
1407 }
```

εδω κανουμε ενα memcpy το μηνυμα σε μια τοπικη μεταβλητη και το βαζουμε αυτο στην ουρα με τα μηνύματα που λαβαμε απο αυτο τον τυπο και καλουμε την receiveAggMinTask, και οι 3 .receive που γραψαμε ειναι ακριβως το ιδιο απλα αλλαζουν οι τυποι και το task για να ειναι αντιστοιχεί για το μηνυμα που λαβαμε

Τώρα για το task που καλούμε κανουμε deque το μήνυμα που λαβαμε εχουμε εναν ελεγχο να δουμε αν το μεγεθος του ειναι ισο με αυτο που περιμενουμε να λαβουμε και αν ισχυει περνουμε τα περιεχομενα του πακετου αυτου, εχουμε εναν ελεγχο για να δουμε οτι ολα πηγαν καλα και μετα αν η τιμη που μας σταλθηκε ειναι μικροτερη απο αυτην που εχουμε αποθηκευμενη την πανω γραφουμε αλλιως δεν κανουμε κατι.

```
1453 task void receiveAggMinTask(){
1454     uint8_t len;
1455     //uint16_t msource;
1456     message_t radioAggMinRecPkt;
1457     AggregationMin* mpkt;
1458
1459     dbg("Min","ReceiveAggMinTask():received msg...\n");
1460     radioAggMinRecPkt = call AggMinReceiveQueue.dequeue();
1461     len = call AggMinPacket.payloadLength(&radioAggMinRecPkt);
1462     dbg("Min","ReceiveAggMinTask(): len=%u \n",len);
1463     if(len == sizeof(AggregationMin)){
1464         mpkt = (AggregationMin*) (call AggMinPacket.getPayload(&radioAggMinRecPkt,len));
1465         if(mpkt==NULL){
1466             dbg("Min","receiveAggMinTask(): No valid payload... \n");
1467             return;
1468         }
1469         dbg("Min","receiveAggMinTask(): minVal= %u \n", mpkt->minVal);
1470         if(mpkt->minVal < agg_min){
1471             agg_min = mpkt->minVal;
1472             dbg("Min","Inside the ReceiveAggMinTask(): New agg_min = %u \n", agg_min);
1473         }
1474     } //dbg("Sample","New sample = %u \n", sample);
1475 }
```

Με ακριβως ιδιο τροπο δουλευουν και τα αλλα δυο tasks απλα με αλλαγμενουν τυπους και χωρις ελεγχο για τις τιμες που λαμβανουμε γιατι δεν χρειαζεται απλα τις προσθετουμε στις αντιστοιχες μεταβλητες

3. Εξήγηση για την δομή των μηνυμάτων

Εδώ δεν χρειάζεται να το αναλύσουμε υπερβολικά αφού και μόνο ο τίτλος κάθε μεταβλητής λέει γιατί χρησιμοποιείται.

AggregationMin	
minVal	στέλνουμε την μικρότερη τιμή που έχουμε στον κομβό μας
AggregationSum	
sum	στελνουμε το άθροισμα που έχουμε υπολογίσει
AggregationAvg	
sum	στελνουμε το άθροισμα που έχουμε υπολογίσει
count	στελνουμε τον αριθμό των κομβών που έχουμε υπολογίσει
RoutingMsg	
depth	το βάθος στο δέντρο το οποίο είμαστε
aggType	τον τύπο query που θα εκτελέσουμε

Ποιοι πολύ ενδιαφέρον έχει να δούμε το μέγεθος κάθε μεταβλητής στα πακέτα που έχουμε ορίσει

AggregationMin		
nx_uint8_t	minVal	επειδή οι τιμές είναι από 1-60 μας καλύπτει το 8bit που φτάνει μέχρι το 255
AggregationSum		
nx_uint16_t	sum	αν πάρουμε την υποθεση που μας δίνεται στην εκφώνηση ότι ο μέγιστος αριθμός κομβών είναι 100 και όλοι έχουν επιλεξει 60 έχουμε 6.000 σαν άθροισμα που δεν χωράει στο uint8 οπότε διαλέξαμε το ακριβώς επόμενο
AggregationAvg		
nx_uint16_t	sum	ομοίως με πάνω
nx_uint8_t	count	πάλι με την υποθεση των 100 κομβών χωράει στο 8bit
RoutingMsg		
nx_uint8_t	depth	ακόμα και στην χειρότερη περίπτωση που έχουμε μια γραμμή χωράει
nx_uint8_t	aggType	έχουμε τιμές 1-3 άρα σίγουρα χωράει θα χώραγε και σε 4 αλλά δεν είχαμε αυτήν την επιλογή

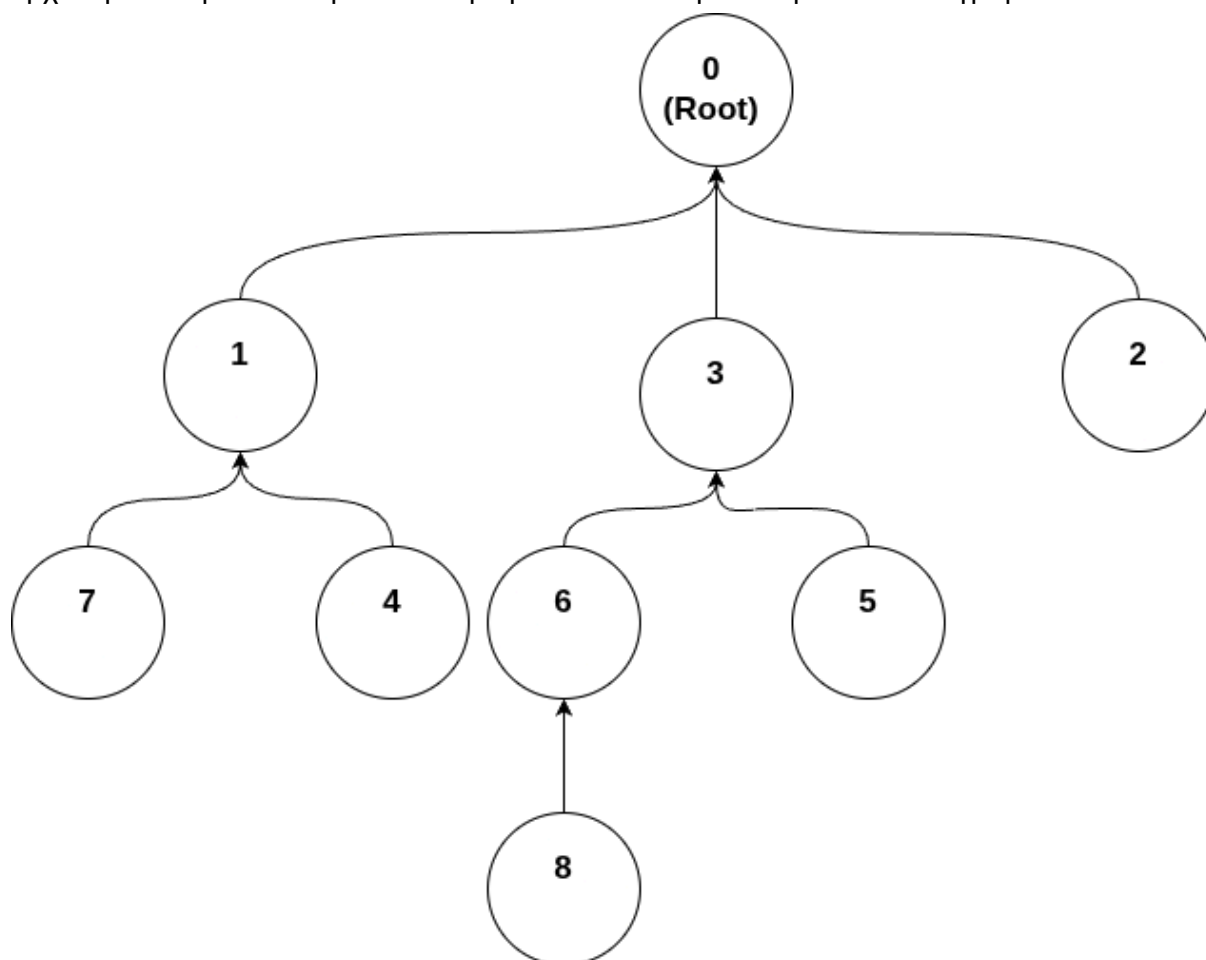
4.Προσομοιώσεις

Σε κάθε προσομοίωσεις θα δούμε τις 3 πρώτες εποχές για να βεβαιωθούμε ότι τρέχει καλά.
Όλες οι προσομοιώσεις τρέχουν το topology2.txt που μας δόθηκε
Επίσης θα βρείτε στο zip τα logs όλων των προσομοιώσεων που βρίσκονται στην αναφορά

Query: Min

```
0:0:10.202499377 DEBUG (3): NodeID 3: parentID= 0 , curdepth= 1 , aggType= 1
0:0:10.202499377 DEBUG (1): NodeID 1: parentID= 0 , curdepth= 1 , aggType= 1
0:0:10.202499377 DEBUG (2): NodeID 2: parentID= 0 , curdepth= 1 , aggType= 1
0:0:10.406524635 DEBUG (7): NodeID 7: parentID= 1 , curdepth= 2 , aggType= 1
0:0:10.406524635 DEBUG (4): NodeID 4: parentID= 1 , curdepth= 2 , aggType= 1
0:0:10.406570411 DEBUG (6): NodeID 6: parentID= 3 , curdepth= 2 , aggType= 1
0:0:10.406570411 DEBUG (5): NodeID 5: parentID= 3 , curdepth= 2 , aggType= 1
0:0:10.606796264 DEBUG (8): NodeID 8: parentID= 6 , curdepth= 3 , aggType= 1
```

Αρχικά βλέπουμε το δέντρο που παρειγε και κατω το βλέπουμε και σε διαγραμμα

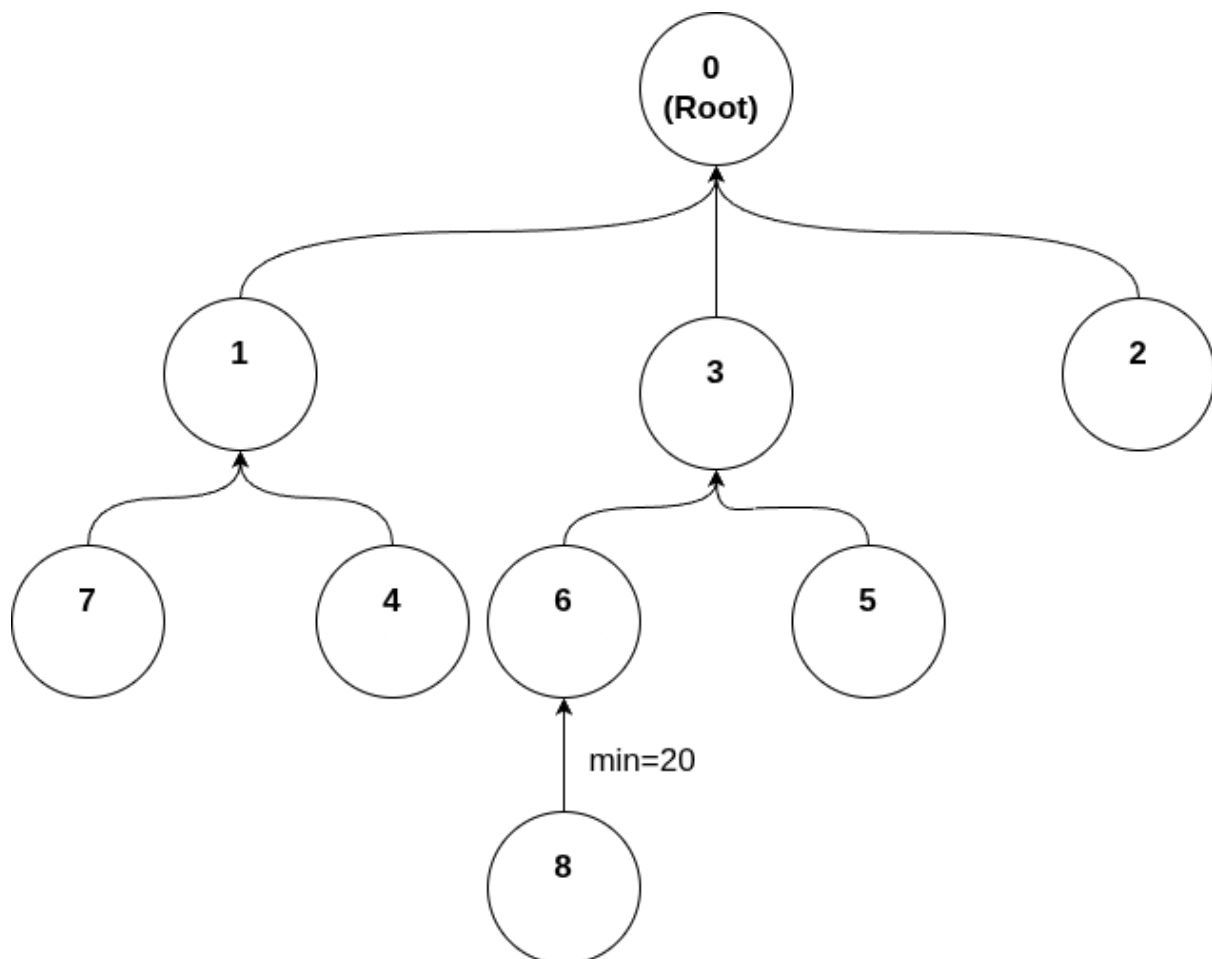


Επειτα ο 8 παίρνει την μετρηση του και την στέλνει στον 6 που την λαμβανει επιτυχως

```

0:0:29.304687510 DEBUG (8): NodeID= 8 : AggregationMin sample= 20 , agg_min= 255
0:0:29.304687510 DEBUG (8): NodeID= 8 : AggregationMin value= 20
0:0:29.304687510 DEBUG (8): SendAggMinTask() posted!!
0:0:29.304687510 DEBUG (8): AggregationMin enqueued successfully in SendingQueue!!!
0:0:29.304687520 DEBUG (8): SendAggMinTask(): Starting....
0:0:29.304687520 DEBUG (8): SendAggMinTask(): Send returned success!!!
0:0:29.306533823 DEBUG (6): ### AggMinReceive.receive() start #####
0:0:29.306533823 DEBUG (6): Something received!!! from 8
0:0:29.306533823 DEBUG (6): posting receiveAggMinTask()!!!!
0:0:29.306533833 DEBUG (6): ReceiveAggMinTask():received msg...
0:0:29.306533833 DEBUG (6): ReceiveAggMinTask(): len=1
0:0:29.306533833 DEBUG (6): receiveAggMinTask(): minVal= 20
0:0:29.306533833 DEBUG (6): Inside the ReceiveAggMinTask(): New agg_min = 20
0:0:29.306701669 DEBUG (8): Inside the AggMinAMSend.sendDone()
0:0:29.306701669 DEBUG (8): A AggregationMin package sent... True|

```



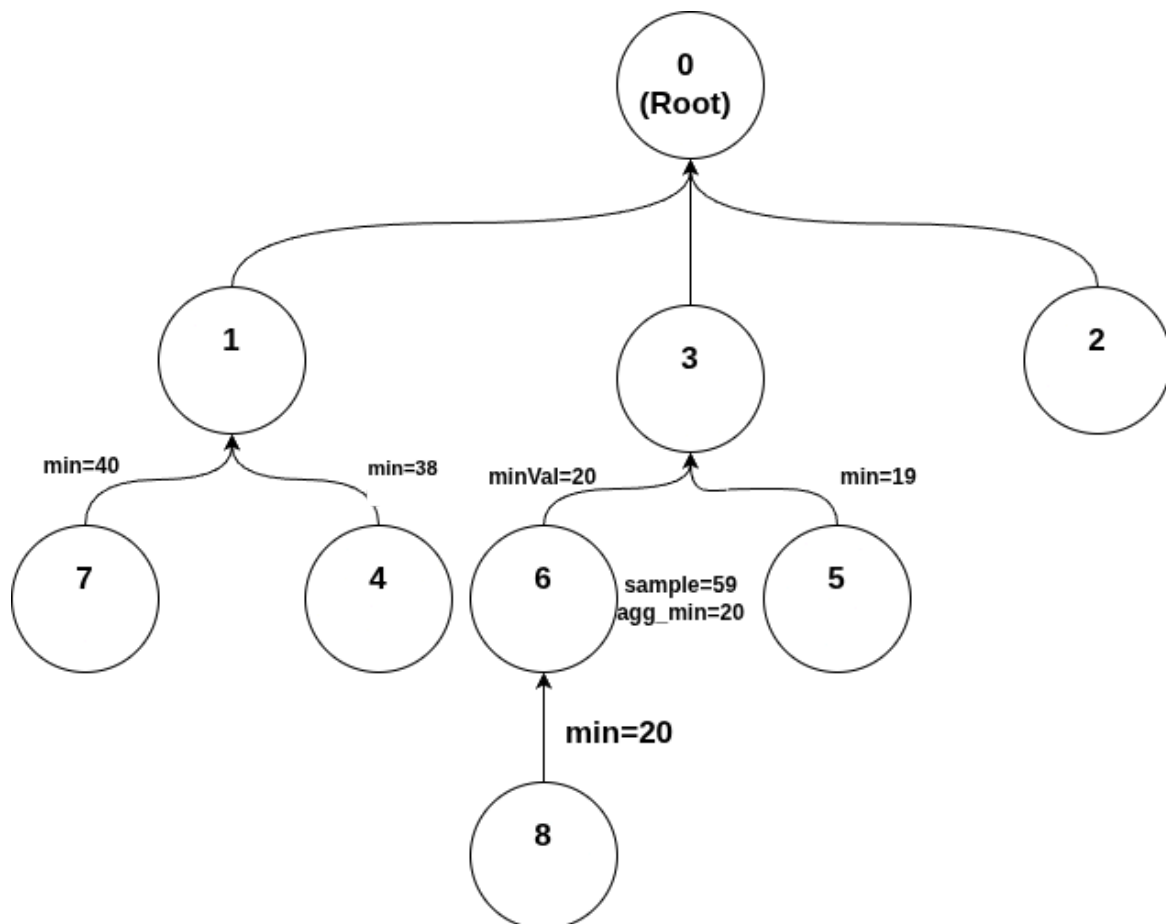
Επειτα είναι η σειρά του 4, 7, 6, 5 να παρουν μετρηση και να την στείλουν. Αρχίζει ο 4 στην προσομοίωση μας.

Παρατηρούμε ότι στέλνει ο 4 την τιμή 38 όμοια ο 5 την τιμή 19 ο 6 την τιμή 59 άλλα έχει λάβει από τον 8 την τιμή 20 αρα θα στείλει την μικρότερη που είναι 20 του 8 .Τέλος ο 7 την τιμή 40 οπότε έχουμε το παρακάτω.

```

0:0:29.375000010 DEBUG (4): NodeID= 4 : AggregationMin sample= 38 , agg_min= 255
0:0:29.375000010 DEBUG (4): NodeID= 4 : AggregationMin value= 38
0:0:29.375000010 DEBUG (4): SendAggMinTask() posted!!
0:0:29.375000010 DEBUG (4): AggregationMin enqueued successfully in SendingQueue!!!
0:0:29.375000020 DEBUG (4): SendAggMinTask(): Starting....
0:0:29.375000020 DEBUG (4): SendAggMinTask(): Send returned success!!!
0:0:29.376953135 DEBUG (5): NodeID= 5 : AggregationMin sample= 19 , agg_min= 255
0:0:29.376953135 DEBUG (5): NodeID= 5 : AggregationMin value= 19
0:0:29.376953135 DEBUG (5): SendAggMinTask() posted!!
0:0:29.376953135 DEBUG (5): AggregationMin enqueued successfully in SendingQueue!!!
0:0:29.376953145 DEBUG (5): SendAggMinTask(): Starting....
0:0:29.376953145 DEBUG (5): SendAggMinTask(): Send returned success!!!
0:0:29.378906260 DEBUG (6): NodeID= 6 : AggregationMin sample= 59 , agg_min= 20
0:0:29.378906260 DEBUG (6): NodeID= 6 : AggregationMin value= 20
0:0:29.378906260 DEBUG (6): SendAggMinTask() posted!!
0:0:29.378906260 DEBUG (6): AggregationMin enqueued successfully in SendingQueue!!!
0:0:29.378906270 DEBUG (6): SendAggMinTask(): Starting....
0:0:29.378906270 DEBUG (6): SendAggMinTask(): Send returned success!!!
0:0:29.380859385 DEBUG (7): NodeID= 7 : AggregationMin sample= 40 , agg_min= 255
0:0:29.380859385 DEBUG (7): NodeID= 7 : AggregationMin value= 40
0:0:29.380859385 DEBUG (7): SendAggMinTask() posted!!
0:0:29.380859385 DEBUG (7): AggregationMin enqueued successfully in SendingQueue!!!
0:0:29.380859395 DEBUG (7): SendAggMinTask(): Starting....
0:0:29.380859395 DEBUG (7): SendAggMinTask(): Send returned success!!!

```

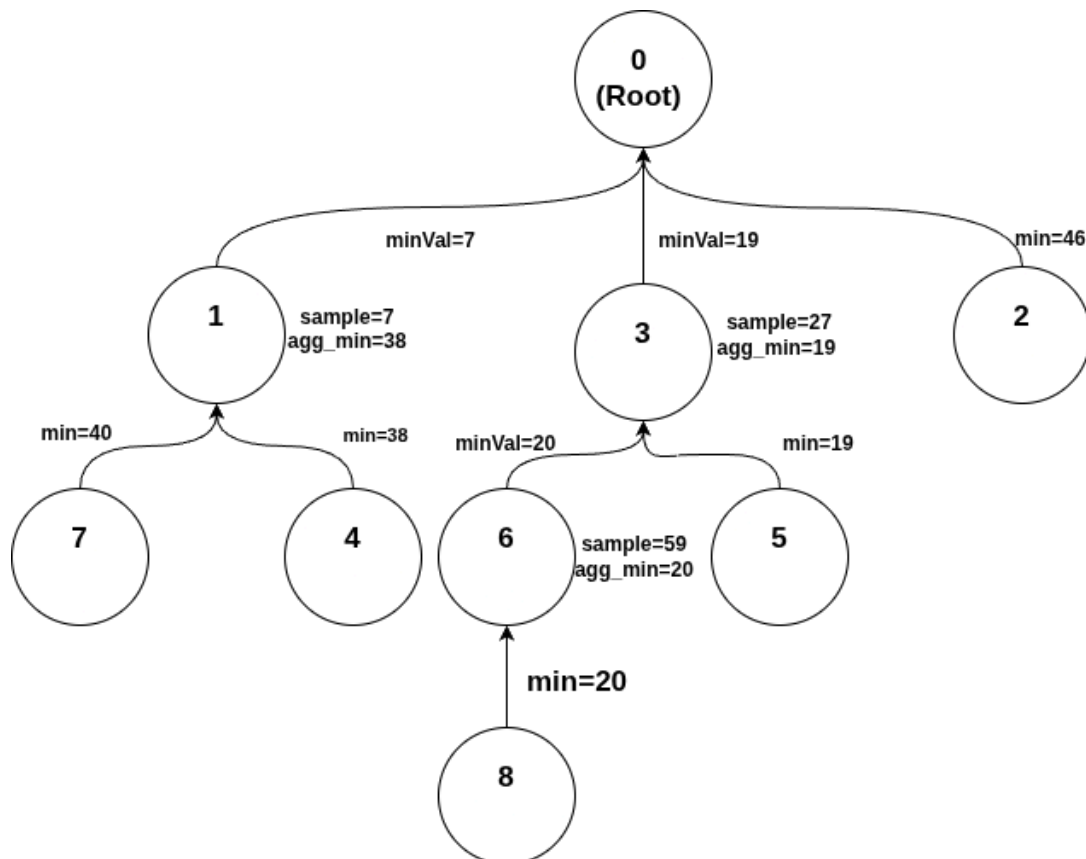


Ομοία με την διαδικασία του 6 που απαιτούν συγκρίσεις μεταξύ του sample και της τιμής που λαμβάνει ο αισθητήρας συμβαίνει και για τους κομβούς 1,3,2

```

0:0:29.381500238 DEBUG (3): ### AggMinReceive.receive() start #####
0:0:29.381500238 DEBUG (3): Something received!!! from 5
0:0:29.381500238 DEBUG (3): posting receiveAggMinTask()!!!!
0:0:29.381500248 DEBUG (3): ReceiveAggMinTask():received msg...
0:0:29.381500248 DEBUG (3): ReceiveAggMinTask(): len=1
0:0:29.381500248 DEBUG (3): receiveAggMinTask(): minVal= 19
0:0:29.381500248 DEBUG (3): Inside the ReceiveAggMinTask(): New agg_min = 19
0:0:29.381668083 DEBUG (5): Inside the AggMinAMSend.sendDone()
0:0:29.381668083 DEBUG (5): A AggregationMin package sent... True
0:0:29.384857140 DEBUG (1): ### AggMinReceive.receive() start #####
0:0:29.384857140 DEBUG (1): Something received!!! from 4
0:0:29.384857140 DEBUG (1): posting receiveAggMinTask()!!!!
0:0:29.384857150 DEBUG (1): ReceiveAggMinTask():received msg...
0:0:29.384857150 DEBUG (1): ReceiveAggMinTask(): len=1
0:0:29.384857150 DEBUG (1): receiveAggMinTask(): minVal= 38
0:0:29.384857150 DEBUG (1): Inside the ReceiveAggMinTask(): New agg_min = 38
0:0:29.385024986 DEBUG (4): Inside the AggMinAMSend.sendDone()
0:0:29.385024986 DEBUG (4): A AggregationMin package sent... True
0:0:29.385635357 DEBUG (3): ### AggMinReceive.receive() start #####
0:0:29.385635357 DEBUG (3): Something received!!! from 6
0:0:29.385635357 DEBUG (3): posting receiveAggMinTask()!!!!
0:0:29.385635367 DEBUG (3): ReceiveAggMinTask():received msg...
0:0:29.385635367 DEBUG (3): ReceiveAggMinTask(): len=1
0:0:29.385635367 DEBUG (3): receiveAggMinTask(): minVal= 20

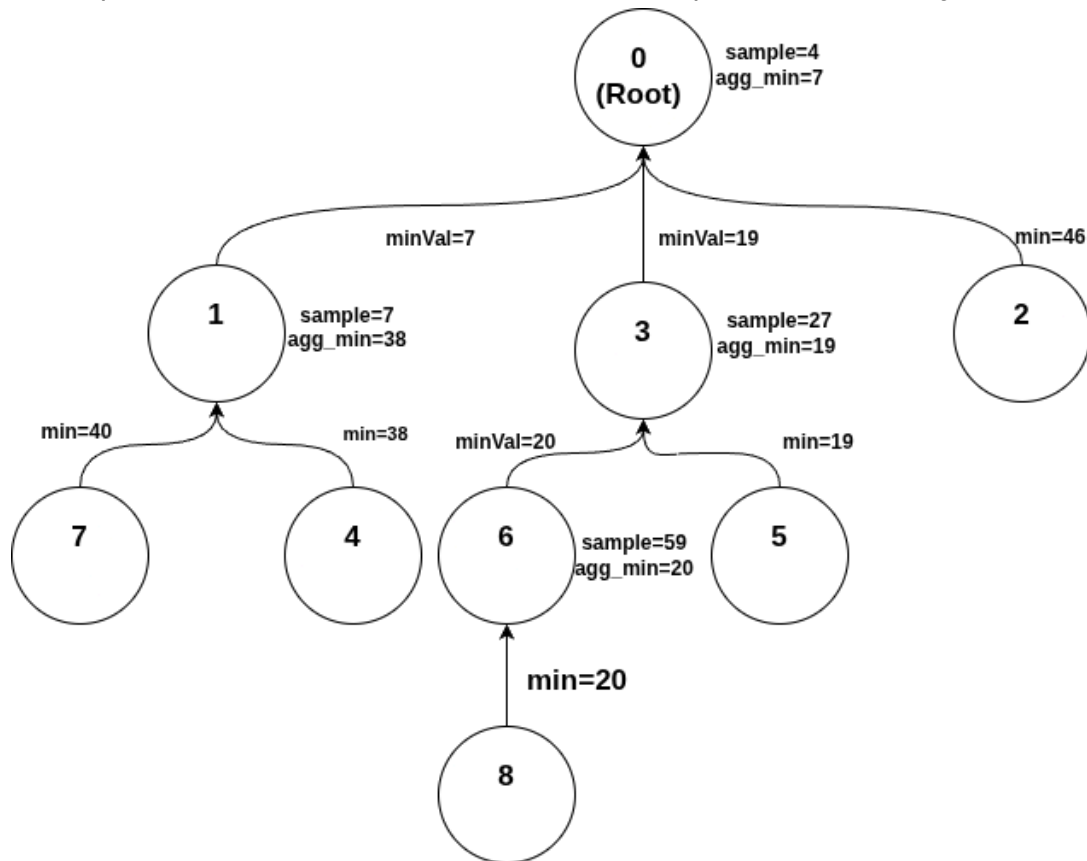
```



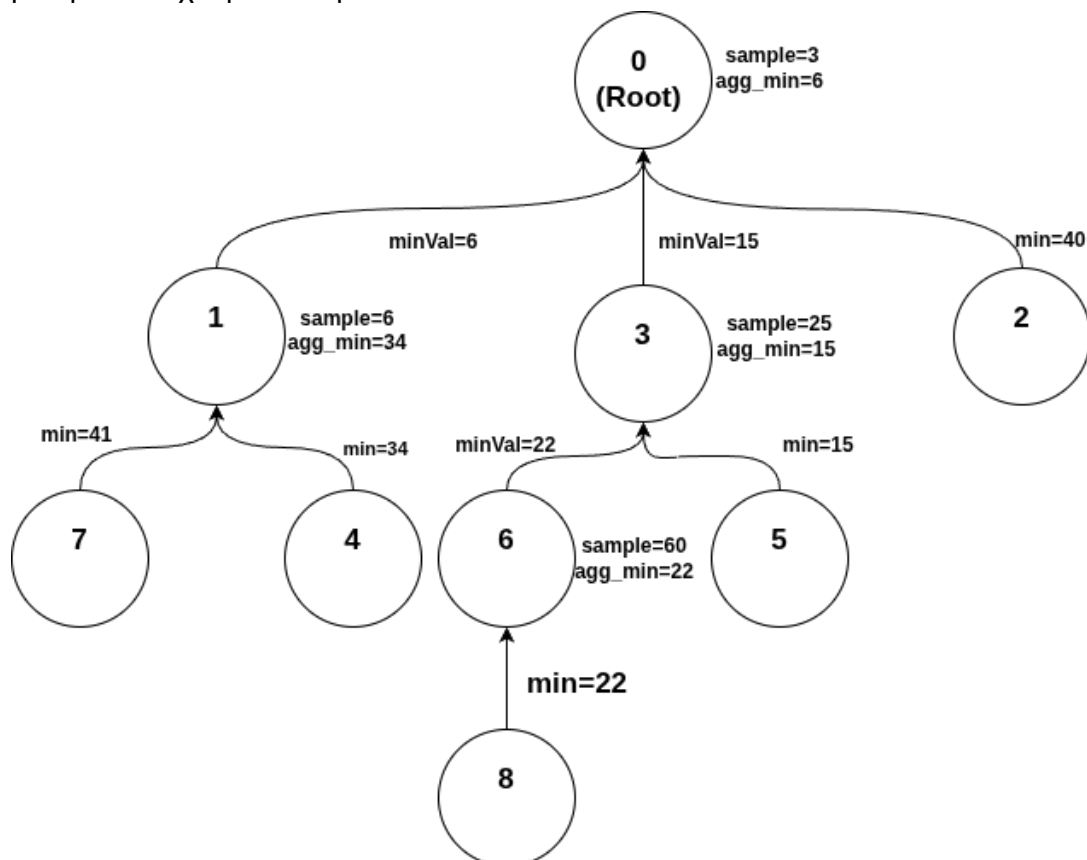
και τέλος έχουμε την ίδια διαδικασία για των κομβο 0 συγκριση του sample του με το μήνυμα του 1,3,2

```
0:0:29.469131436 DEBUG (0): ### AggMinReceive.receive() start #####
0:0:29.469131436 DEBUG (0): Something received!!! from 1
0:0:29.469131436 DEBUG (0): posting receiveAggMinTask()!!!!
0:0:29.469131446 DEBUG (0): ReceiveAggMinTask():received msg...
0:0:29.469131446 DEBUG (0): ReceiveAggMinTask(): len=1
0:0:29.469131446 DEBUG (0): receiveAggMinTask(): minVal= 7
0:0:29.469131446 DEBUG (0): Inside the ReceiveAggMinTask(): New agg_min = 7
0:0:29.469299282 DEBUG (1): Inside the AggMinAMSend.sendDone()
0:0:29.469299282 DEBUG (1): A AggregationMin package sent... True
0:0:29.471496546 DEBUG (0): ### AggMinReceive.receive() start #####
0:0:29.471496546 DEBUG (0): Something received!!! from 3
0:0:29.471496546 DEBUG (0): posting receiveAggMinTask()!!!!
0:0:29.471496556 DEBUG (0): ReceiveAggMinTask():received msg...
0:0:29.471496556 DEBUG (0): ReceiveAggMinTask(): len=1
0:0:29.471496556 DEBUG (0): receiveAggMinTask(): minVal= 19
0:0:29.471664392 DEBUG (3): Inside the AggMinAMSend.sendDone()
0:0:29.471664392 DEBUG (3): A AggregationMin package sent... True
0:0:29.536132822 DEBUG (0): NodeID= 0 : AggregationMin sample= 4 , agg_min= 7
0:0:29.536132822 DEBUG (0): ***//////////\***
0:0:29.536132822 DEBUG (0): AGG RESULT epoch=1 MIN=4
0:0:29.536132822 DEBUG (0): ***\\//\//////////***
```

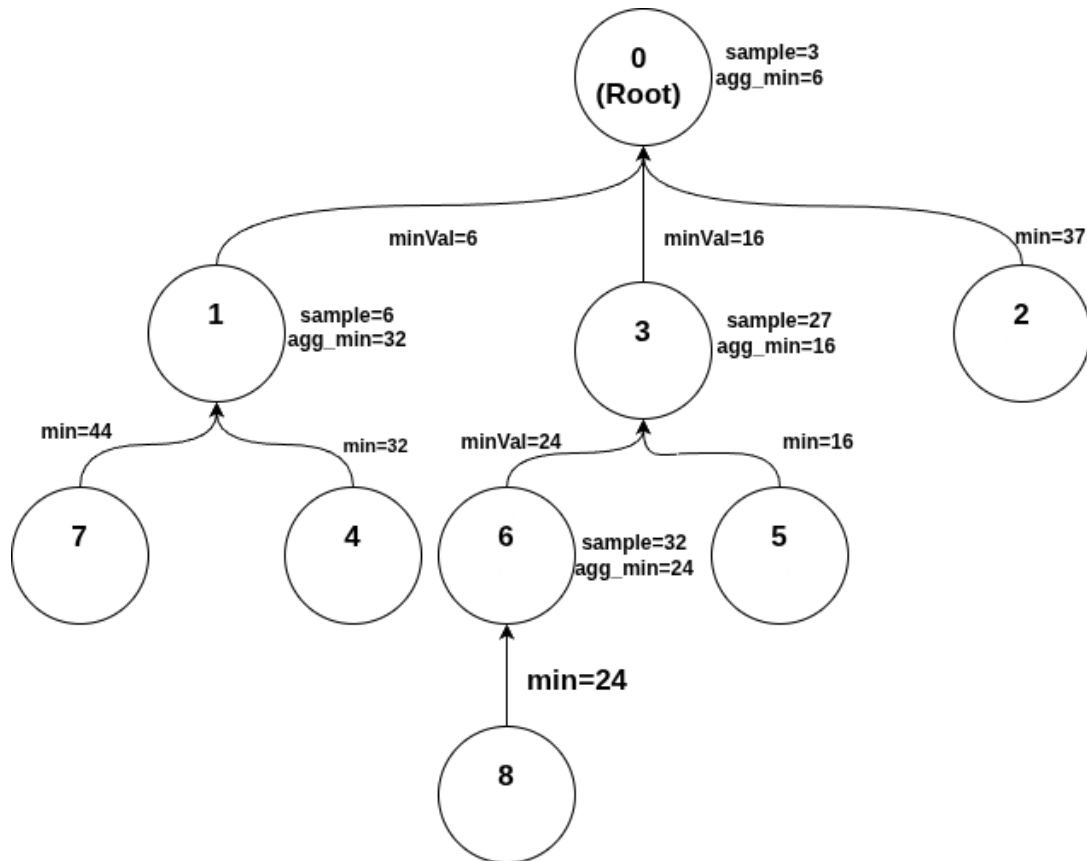
Παρατηρούμε ότι σε αυτό το epoch όλοι έλαβαν τα μηνύματα όλων χωρίς να χάνονται



για epoch 2 έχουμε το παρακάτω



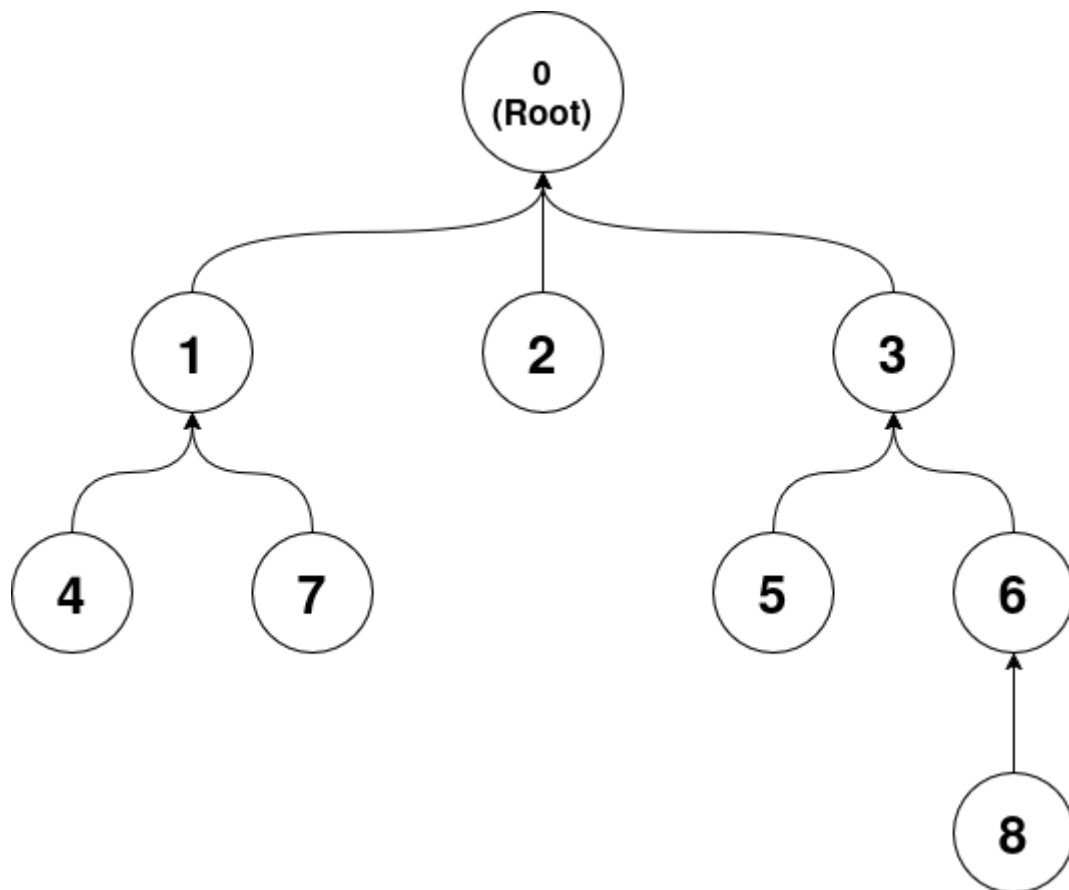
για epoch 3 έχουμε το παρακάτω



Query: Sum

```
0:0:10.201019283 DEBUG (3): NodeID 3: parentID= 0 , curdepth= 1 , aggType= 2
0:0:10.201019283 DEBUG (1): NodeID 1: parentID= 0 , curdepth= 1 , aggType= 2
0:0:10.201019283 DEBUG (2): NodeID 2: parentID= 0 , curdepth= 1 , aggType= 2
0:0:10.398376478 DEBUG (5): NodeID 5: parentID= 3 , curdepth= 2 , aggType= 2
0:0:10.398376478 DEBUG (6): NodeID 6: parentID= 3 , curdepth= 2 , aggType= 2
0:0:10.398635876 DEBUG (4): NodeID 4: parentID= 1 , curdepth= 2 , aggType= 2
0:0:10.398635876 DEBUG (7): NodeID 7: parentID= 1 , curdepth= 2 , aggType= 2
0:0:10.595764173 DEBUG (8): NodeID 8: parentID= 6 , curdepth= 3 , aggType= 2
```

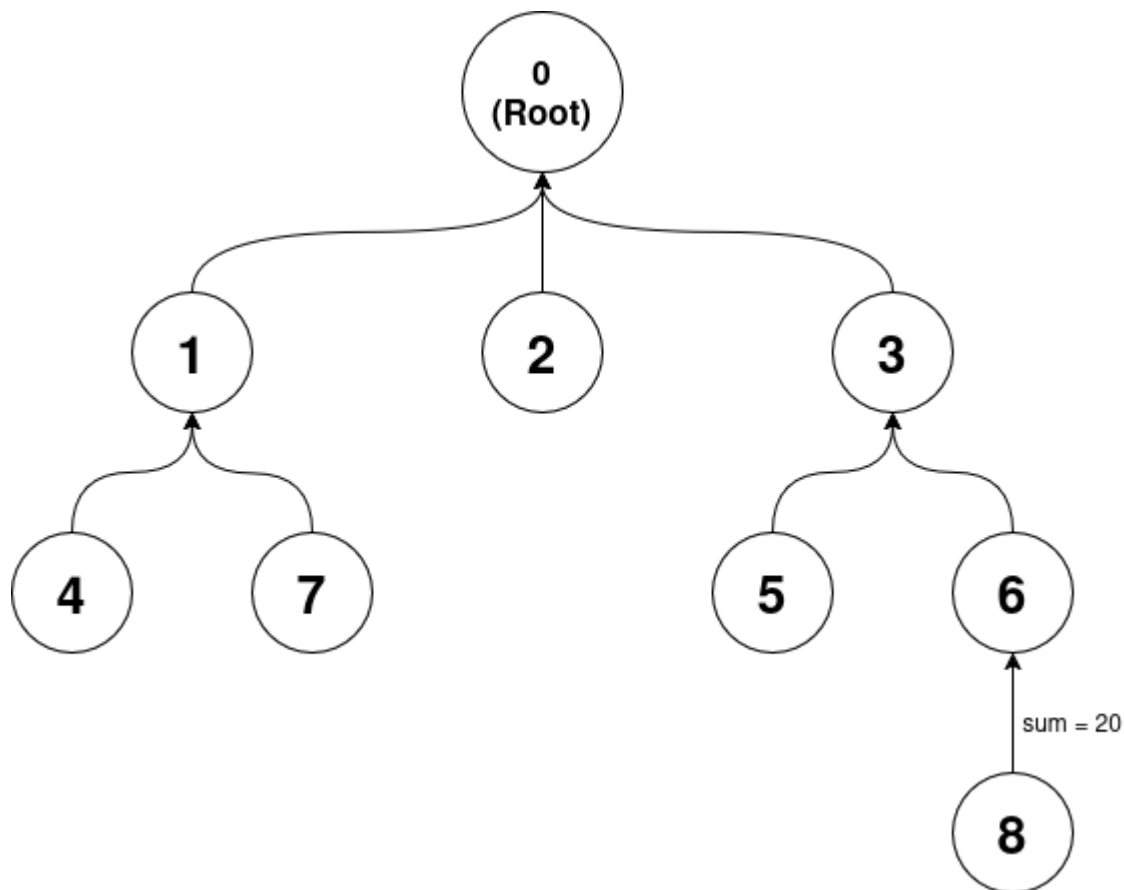
Αρχικά βλέπουμε το δέντρο που παρείχε και κάτω το βλέπουμε και σε διαγράμα



Επειτα ο 8 παίρνει την μετρηση του και την στέλνει στον 6 που την λαμβανει επιτυχως

```

0:0:29.300781260 DEBUG (8): NodeID= 8 : AggregationSum sample= 20 , sumBefore= 0
0:0:29.300781260 DEBUG (8): NodeID= 8 : AggregationSum sumAfter= 20
0:0:29.300781260 DEBUG (8): NodeID= 8 : AggregationSum value= 20
0:0:29.300781260 DEBUG (8): SendAggSumTask() posted!!
0:0:29.300781260 DEBUG (8): AggregationSum enqueued successfully in SendingQueue!!!
0:0:29.300781270 DEBUG (8): SendAggSumTask(): Starting...
0:0:29.300781270 DEBUG (8): SendAggSumTask(): Send returned success!!!
0:0:29.305664054 DEBUG (6): ### AggSumReceive.receive() start #####
0:0:29.305664054 DEBUG (6): Something received!!! from 8
0:0:29.305664054 DEBUG (6): posting receiveAggSumTask()!!!!
0:0:29.305664064 DEBUG (6): ReceiveAggSumTask():received msg...
0:0:29.305664064 DEBUG (6): ReceiveAggSumTask(): len=2
0:0:29.305664064 DEBUG (6): receiveAggSumTask(): sum= 20
0:0:29.305664064 DEBUG (6): Inside the ReceiveAggSumTask(): New agg_sum = 20
0:0:29.305831900 DEBUG (8): Inside the AggSumAMSend.sendDone()
0:0:29.305831900 DEBUG (8): A AggregationSum package sent... True
  
```

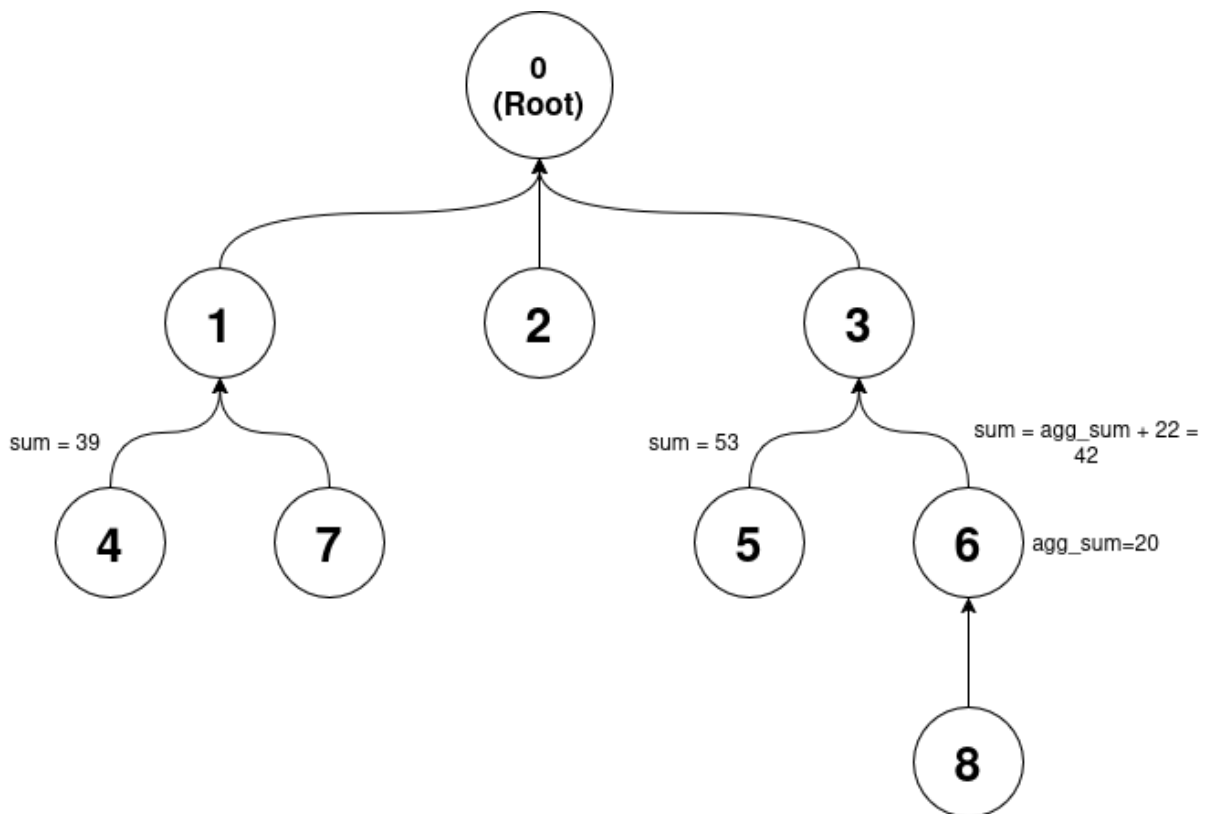


Επειτα είναι η σειρά του 4, 7, 5, 6 να παρουν μετρηση και να την στείλουν. Αρχίζει ο 6 στην προσομοίωση μας.

Είχε λαβει απο τον 8 το 20 και περνει μετρηση 22 τα προσθετει βγαζει 42 και ετοιμαζεται να το στείλει, ενω ταυτοχρονα ο 4 βγαζει μετρηση 39 και το ετοιμαζει να το στείλει και ο 5 βγαζει 53, πιο κατω βγαζει και ο 7 αλλα πριν απο αυτο εχουν προλαβει να γινουν καποια receive

```

0:0:29.375000010 DEBUG (6): NodeID= 6 : AggregationSum sample= 22 , sumBefore= 20
0:0:29.375000010 DEBUG (6): NodeID= 6 : AggregationSum sumAfter= 42
0:0:29.375000010 DEBUG (6): NodeID= 6 : AggregationSum value= 42
0:0:29.375000010 DEBUG (6): SendAggSumTask() posted!!
0:0:29.375000010 DEBUG (6): AggregationSum enqueued successfully in SendingQueue!!!
0:0:29.375000020 DEBUG (6): SendAggSumTask(): Starting....
0:0:29.375000020 DEBUG (6): SendAggSumTask(): Send returned success!!!
0:0:29.376953135 DEBUG (4): NodeID= 4 : AggregationSum sample= 39 , sumBefore= 0
0:0:29.376953135 DEBUG (4): NodeID= 4 : AggregationSum sumAfter= 39
0:0:29.376953135 DEBUG (4): NodeID= 4 : AggregationSum value= 39
0:0:29.376953135 DEBUG (4): SendAggSumTask() posted!!
0:0:29.376953135 DEBUG (4): AggregationSum enqueued successfully in SendingQueue!!!
0:0:29.376953145 DEBUG (4): SendAggSumTask(): Starting....
0:0:29.376953145 DEBUG (4): SendAggSumTask(): Send returned success!!!
0:0:29.377929698 DEBUG (5): NodeID= 5 : AggregationSum sample= 53 , sumBefore= 0
0:0:29.377929698 DEBUG (5): NodeID= 5 : AggregationSum sumAfter= 53
0:0:29.377929698 DEBUG (5): NodeID= 5 : AggregationSum value= 53
0:0:29.377929698 DEBUG (5): SendAggSumTask() posted!!
0:0:29.377929698 DEBUG (5): AggregationSum enqueued successfully in SendingQueue!!!
0:0:29.377929708 DEBUG (5): SendAggSumTask(): Starting....
0:0:29.377929708 DEBUG (5): SendAggSumTask(): Send returned success!!!
0:0:29.377929708 DEBUG (3): ### AggSumReceive receive() start #####
  
```

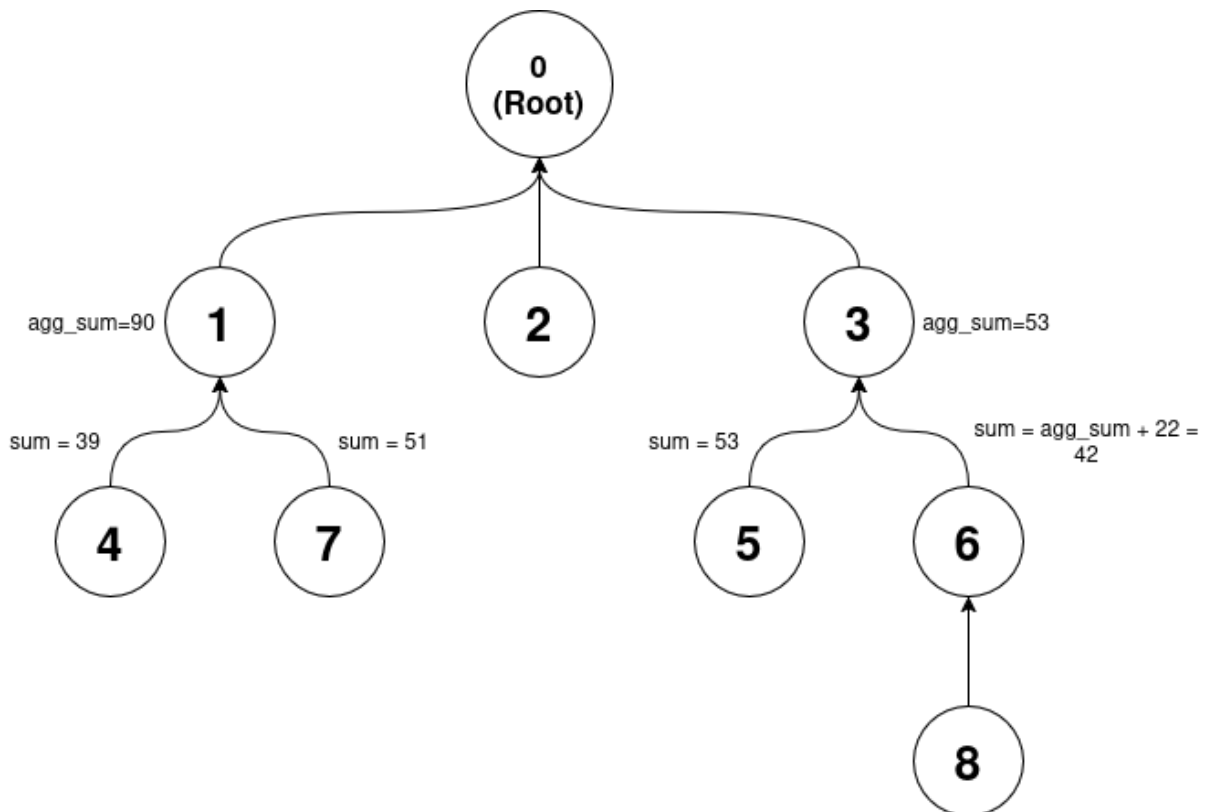


Ο 3 λαμβάνει από τον 5 το 53 και το προσθέτει στο άθροισμα του που δεν είχε κάτι άρα 53. Μετά ο 1 λαμβάνει από το 4 το 39 άρα το $\text{agg_sum} = 39$. Και μετά βλέπουμε τον 7 να περνει μέτρηση το 51 να το στέλνει να το λαμβάνει ο 1 και να το προσθέτει στο agg_sum του άρα $\text{agg_sum} = \text{agg_sum} + 51 = 39 + 51 = 90$

```

0:0:29.380004891 DEBUG (3): Something received!!! from 5
0:0:29.380004891 DEBUG (3): posting receiveAggSumTask()!!!!
0:0:29.380004901 DEBUG (3): ReceiveAggSumTask():received msg...
0:0:29.380004901 DEBUG (3): ReceiveAggSumTask(): len=2
0:0:29.380004901 DEBUG (3): receiveAggSumTask(): sum= 53
0:0:29.380004901 DEBUG (3): Inside the ReceiveAggSumTask(): New agg_sum = 53
0:0:29.380172736 DEBUG (5): Inside the AggSumAMSend.sendDone()
0:0:29.380172736 DEBUG (5): A AggregationSum package sent... True
0:0:29.381805412 DEBUG (1): ### AggSumReceive.receive() start #####
0:0:29.381805412 DEBUG (1): Something received!!! from 4
0:0:29.381805412 DEBUG (1): posting receiveAggSumTask()!!!!
0:0:29.381805422 DEBUG (1): ReceiveAggSumTask():received msg...
0:0:29.381805422 DEBUG (1): ReceiveAggSumTask(): len=2
0:0:29.381805422 DEBUG (1): receiveAggSumTask(): sum= 39
0:0:29.381805422 DEBUG (1): Inside the ReceiveAggSumTask(): New agg_sum = 39
0:0:29.381835948 DEBUG (7): NodeID= 7 : AggregationSum sample= 51 , sumBefore= 0
0:0:29.381835948 DEBUG (7): NodeID= 7 : AggregationSum sumAfter= 51
0:0:29.381835948 DEBUG (7): NodeID= 7 : AggregationSum value= 51
0:0:29.381835948 DEBUG (7): SendAggSumTask() posted!!
0:0:29.381835948 DEBUG (7): AggregationSum enqueued successfully in SendingQueue!!!
0:0:29.381835958 DEBUG (7): SendAggSumTask(): Starting....
0:0:29.381835958 DEBUG (7): SendAggSumTask(): Send returned success!!!
0:0:29.381973257 DEBUG (4): Inside the AggSumAMSend.sendDone()
0:0:29.381973257 DEBUG (4): A AggregationSum package sent... True
0:0:29.383422823 DEBUG (6): Inside the AggSumAMSend.sendDone()
0:0:29.383422823 DEBUG (6): A AggregationSum package sent... True
0:0:29.384780887 DEBUG (1): ### AggSumReceive.receive() start #####
0:0:29.384780887 DEBUG (1): Something received!!! from 7
0:0:29.384780887 DEBUG (1): posting receiveAggSumTask()!!!!
0:0:29.384780897 DEBUG (1): ReceiveAggSumTask():received msg...
0:0:29.384780897 DEBUG (1): ReceiveAggSumTask(): len=2
0:0:29.384780897 DEBUG (1): receiveAggSumTask(): sum= 51
0:0:29.384780897 DEBUG (1): Inside the ReceiveAggSumTask(): New agg_sum = 90
0:0:29.384948733 DEBUG (7): Inside the AggSumAMSend.sendDone()
:

```

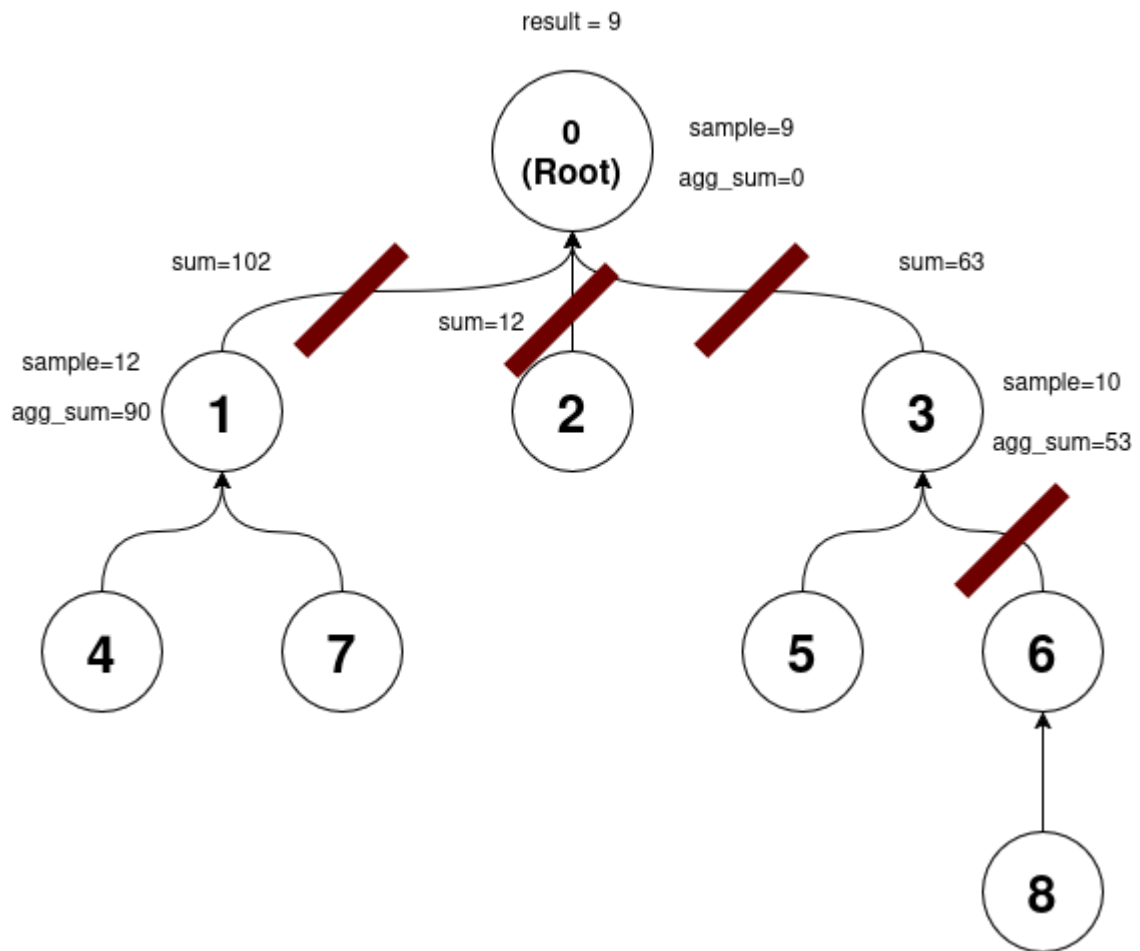


Μετα απο οτι βλεπουμε οτι ο 1 περνει την μετρηση του που ειναι 12 και την προσθετη σε αυτες που εχει λαβει(90) αρα εχει ενα αθροισμα 102

και το στέλνει και ο 3 περνει μετρηση 10 το προσθετη στο 53 που εχει λαβει και βγαζει 63. Εδω να σημειωθει οτι απο οτι φενεται χαθηκε το πακετο απο τον κομβο 6 στον 3 γιαυτο και το sum που εχει ειναι μονο 53 και οχι $53 + 42 = 95$.

Και με την σειρα του ο 2 δεν ειχε παιδια οποτε δεν εχει sum εχει μονο την μετρηση του 12 την οποια και στέλνει. Αλλα επειδη ειμαστε πολυ ατυχει βλεπουμε οτι κανενα απο αυτα δεν τα λαμβανει η ριζα και τυπονει στο τελος το αποτελεσμα μονο της μετρησης της που ειναι το 9. Και εδω τελειωνει η πρωτη εποχη

```
0:0:29.454101572 DEBUG (1): NodeID= 1 : AggregationSum sample= 12 , sumBefore= 90
0:0:29.454101572 DEBUG (1): NodeID= 1 : AggregationSum sumAfter= 102
0:0:29.454101572 DEBUG (1): NodeID= 1 : AggregationSum value= 102
0:0:29.454101572 DEBUG (1): SendAggSumTask() posted!!
0:0:29.454101572 DEBUG (1): AggregationSum enqueued successfully in SendingQueue!!!
0:0:29.454101582 DEBUG (1): SendAggSumTask(): Starting....
0:0:29.454101582 DEBUG (1): SendAggSumTask(): Send returned success!!!
0:0:29.458007822 DEBUG (3): NodeID= 3 : AggregationSum sample= 10 , sumBefore= 53
0:0:29.458007822 DEBUG (3): NodeID= 3 : AggregationSum sumAfter= 63
0:0:29.458007822 DEBUG (3): NodeID= 3 : AggregationSum value= 63
0:0:29.458007822 DEBUG (3): SendAggSumTask() posted!!
0:0:29.458007822 DEBUG (3): AggregationSum enqueued successfully in SendingQueue!!!
0:0:29.458007832 DEBUG (3): SendAggSumTask(): Starting....
0:0:29.458007832 DEBUG (3): SendAggSumTask(): Send returned success!!!
0:0:29.460479742 DEBUG (3): Inside the AggSumAMSend.sendDone()
0:0:29.460479742 DEBUG (3): A AggregationSum package sent... True
0:0:29.460937510 DEBUG (2): NodeID= 2 : AggregationSum sample= 41 , sumBefore= 0
0:0:29.460937510 DEBUG (2): NodeID= 2 : AggregationSum sumAfter= 41
0:0:29.460937510 DEBUG (2): NodeID= 2 : AggregationSum value= 41
0:0:29.460937510 DEBUG (2): SendAggSumTask() posted!!
0:0:29.460937510 DEBUG (2): AggregationSum enqueued successfully in SendingQueue!!!
0:0:29.460937520 DEBUG (2): SendAggSumTask(): Starting....
0:0:29.460937520 DEBUG (2): SendAggSumTask(): Send returned success!!!
0:0:29.461379982 DEBUG (1): Inside the AggSumAMSend.sendDone()
0:0:29.461379982 DEBUG (1): A AggregationSum package sent... True
0:0:29.472183182 DEBUG (2): Inside the AggSumAMSend.sendDone()
0:0:29.472183182 DEBUG (2): A AggregationSum package sent... True
0:0:29.533203135 DEBUG (0): NodeID= 0 : AggregationSum sample= 9 , sumBefore= 0
0:0:29.533203135 DEBUG (0): NodeID= 0 : AggregationSum sumAfter= 9
0:0:29.533203135 DEBUG (0): ***//////////\***
0:0:29.533203135 DEBUG (0): AGG RESULT epoch=1 Sum=9
0:0:29.533203135 DEBUG (0): ***\\//\//\//\//\//\//\//\***
```



Τώρα αρχίζει η δεύτερη εποχή παλι απο τον 8 που περνει μετρηση 17 και το στέλνει στον 6 οπου και αυτος με την σειρα του περνει την δικια του μετρηση $18 + 17 = 35$ και ετοιμαζεται να το στείλει. και ο 4 μετρηση 46 και την στέλνει

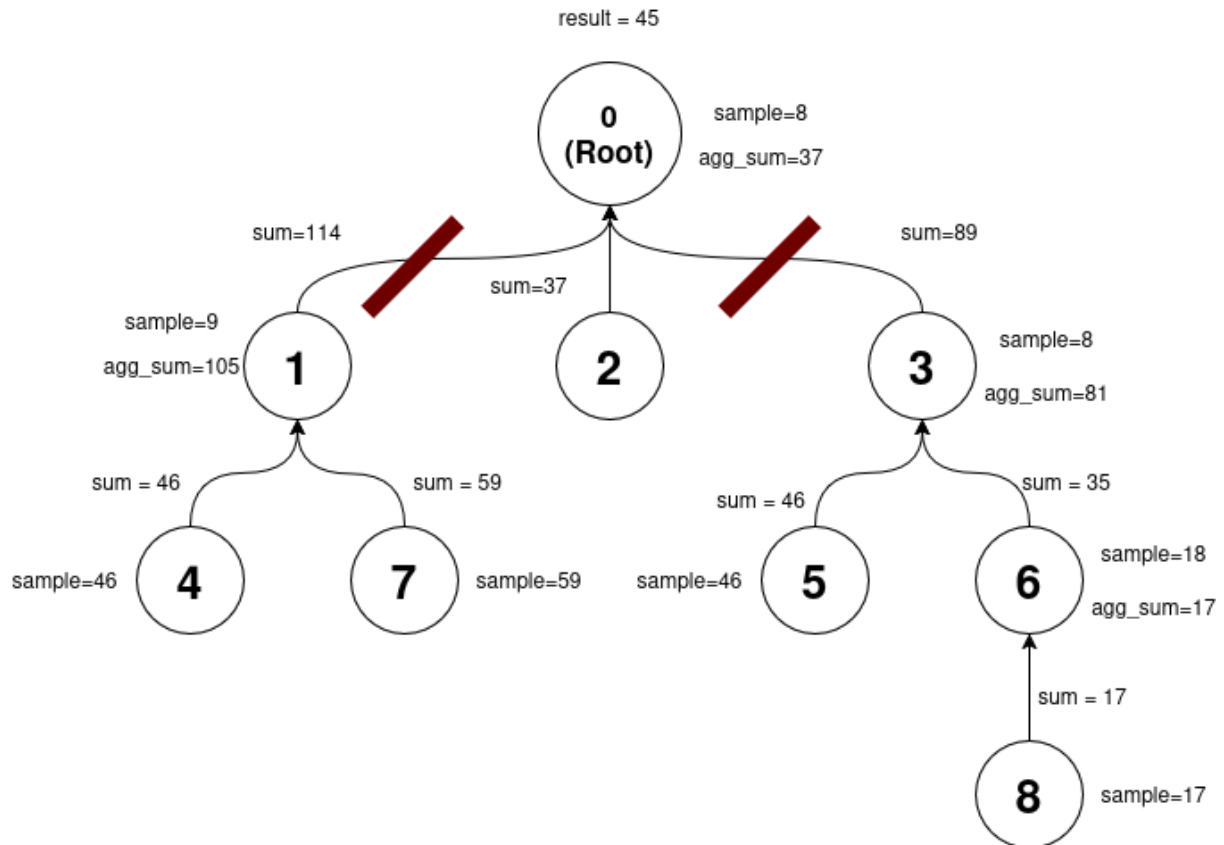
```
0:0:48.832031260 DEBUG (8): NodeID= 8 : AggregationSum sample= 17 , sumBefore= 0
0:0:48.832031260 DEBUG (8): NodeID= 8 : AggregationSum sumAfter= 17
0:0:48.832031260 DEBUG (8): NodeID= 8 : AggregationSum value= 17
0:0:48.832031260 DEBUG (8): SendAggSumTask() posted!!
0:0:48.832031260 DEBUG (8): AggregationSum enqueued successfully in SendingQueue!!!
0:0:48.832031270 DEBUG (8): SendAggSumTask(): Starting....
0:0:48.832031270 DEBUG (8): SendAggSumTask(): Send returned success!!!
0:0:48.837707507 DEBUG (6): #### AggSumReceive.receive() start #####
0:0:48.837707507 DEBUG (6): Something received!!! from 8
0:0:48.837707507 DEBUG (6): posting receiveAggSumTask()!!!!
0:0:48.837707517 DEBUG (6): ReceiveAggSumTask():received msg...
0:0:48.837707517 DEBUG (6): ReceiveAggSumTask(): len=2
0:0:48.837707517 DEBUG (6): receiveAggSumTask(): sum= 17
0:0:48.837707517 DEBUG (6): Inside the ReceiveAggSumTask(): New agg_sum = 17
0:0:48.837875352 DEBUG (8): Inside the AggSumAMSend.sendDone()
0:0:48.837875352 DEBUG (8): A AggregationSum package sent... True
0:0:48.906250010 DEBUG (6): NodeID= 6 : AggregationSum sample= 18 , sumBefore= 17
0:0:48.906250010 DEBUG (6): NodeID= 6 : AggregationSum sumAfter= 35
0:0:48.906250010 DEBUG (6): NodeID= 6 : AggregationSum value= 35
0:0:48.906250010 DEBUG (6): SendAggSumTask() posted!!
0:0:48.906250010 DEBUG (6): AggregationSum enqueued successfully in SendingQueue!!!
0:0:48.906250020 DEBUG (6): SendAggSumTask(): Starting....
0:0:48.906250020 DEBUG (6): SendAggSumTask(): Send returned success!!!
0:0:48.908203135 DEBUG (4): NodeID= 4 : AggregationSum sample= 46 , sumBefore= 0
0:0:48.908203135 DEBUG (4): NodeID= 4 : AggregationSum sumAfter= 46
0:0:48.908203135 DEBUG (4): NodeID= 4 : AggregationSum value= 46
```

Ο 5 βγαζει 46 και το στέλνει το λαμβανει ο 3 και μετα λαμβανει ο 1 του 4 το 46

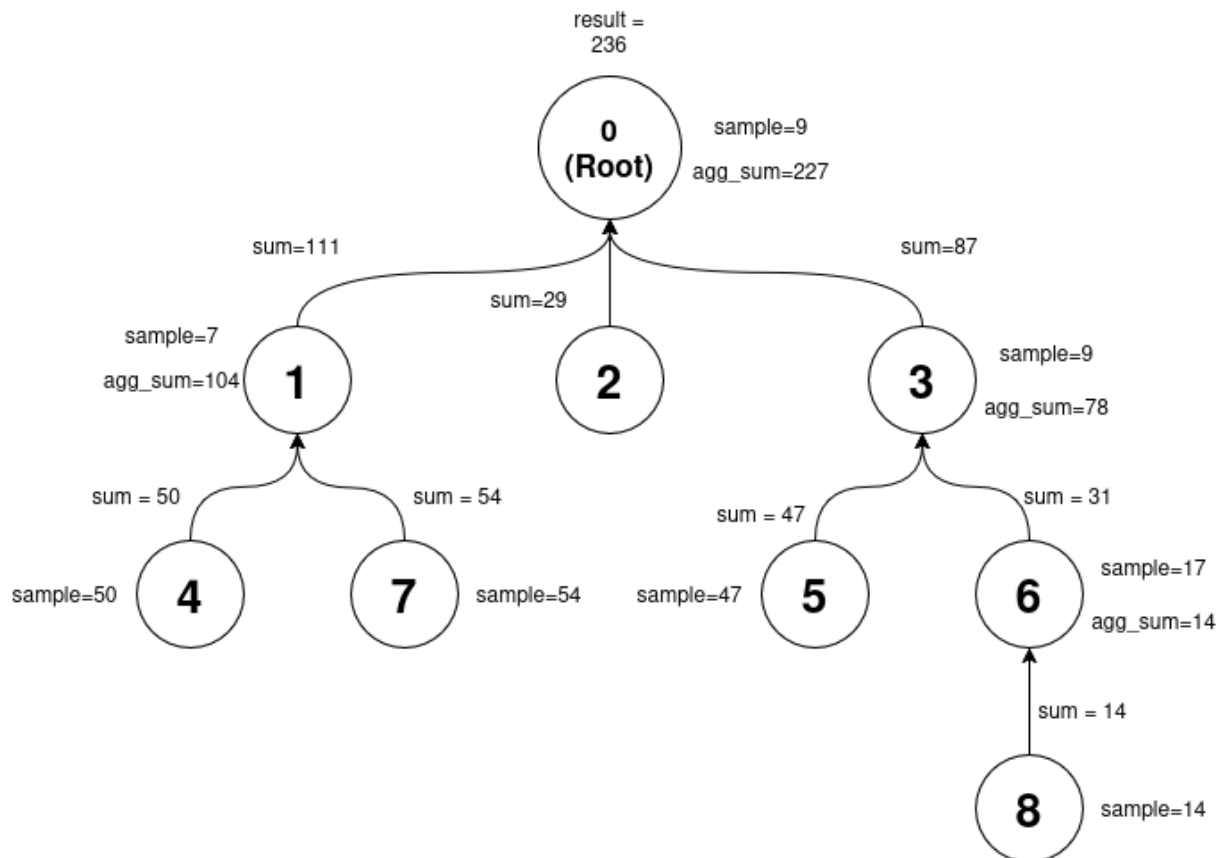
```
0:0:48.908203135 DEBUG (4): SendAggSumTask() posted!!
0:0:48.908203135 DEBUG (4): AggregationSum enqueued successfully in SendingQueue!!!
0:0:48.908203145 DEBUG (4): SendAggSumTask(): Starting....
0:0:48.908203145 DEBUG (4): SendAggSumTask(): Send returned success!!!
0:0:48.909179698 DEBUG (5): NodeID= 5 : AggregationSum sample= 46 , sumBefore= 0
0:0:48.909179698 DEBUG (5): NodeID= 5 : AggregationSum sumAfter= 46
0:0:48.909179698 DEBUG (5): NodeID= 5 : AggregationSum value= 46
0:0:48.909179698 DEBUG (5): SendAggSumTask() posted!!
0:0:48.909179698 DEBUG (5): AggregationSum enqueued successfully in SendingQueue!!!
0:0:48.909179708 DEBUG (5): SendAggSumTask(): Starting....
0:0:48.909179708 DEBUG (5): SendAggSumTask(): Send returned success!!!
0:0:48.911514289 DEBUG (3): ### AggSumReceive.receive() start #####
0:0:48.911514289 DEBUG (3): Something received!!! from 5
0:0:48.911514289 DEBUG (3): posting receiveAggSumTask()!!!!
0:0:48.911514299 DEBUG (3): ReceiveAggSumTask():received msg...
0:0:48.911514299 DEBUG (3): ReceiveAggSumTask(): len=2
0:0:48.911514299 DEBUG (3): receiveAggSumTask(): sum= 46
0:0:48.911514299 DEBUG (3): Inside the ReceiveAggSumTask(): New agg_sum = 46
0:0:48.911682134 DEBUG (5): Inside the AggSumAMSend.sendDone()
0:0:48.911682134 DEBUG (5): A AggregationSum package sent... True
0:0:48.912963859 DEBUG (1): ### AggSumReceive.receive() start #####
0:0:48.912963859 DEBUG (1): Something received!!! from 4
0:0:48.912963859 DEBUG (1): posting receiveAggSumTask()!!!!
0:0:48.912963869 DEBUG (1): ReceiveAggSumTask():received msg...
0:0:48.912963869 DEBUG (1): ReceiveAggSumTask(): len=2
0:0:48.912963869 DEBUG (1): receiveAqqSumTask(): sum= 46
```


επειτα ο 7 περνει την δικια του μετρηση 59 και την στελνει ενω ο 3 λαμβανει την μετρηση του 6 που ηταν 35 την προσθετη με τις υπολοιπες και βγαζει 81 ενω ο 1 ελαβε του 7 το 59 και την προσθετη στις αλλες

```
0:0:48.912963869 DEBUG (1): Inside the ReceiveAggSumTask(): New agg_sum = 46
0:0:48.913085948 DEBUG (7): NodeID= 7 : AggregationSum sample= 59 , sumBefore= 0
0:0:48.913085948 DEBUG (7): NodeID= 7 : AggregationSum sumAfter= 59
0:0:48.913085948 DEBUG (7): NodeID= 7 : AggregationSum value= 59
0:0:48.913085948 DEBUG (7): SendAggSumTask() posted!!
0:0:48.913085948 DEBUG (7): AggregationSum enqueued successfully in SendingQueue!!!
0:0:48.913085958 DEBUG (7): SendAggSumTask(): Starting....
0:0:48.913085958 DEBUG (7): SendAggSumTask(): Send returned success!!!
0:0:48.913131705 DEBUG (4): Inside the AggSumAMSend.sendDone()
0:0:48.913131705 DEBUG (4): A AggregationSum package sent... True
0:0:48.914596529 DEBUG (3): ### AggSumReceive.receive() start #####
0:0:48.914596529 DEBUG (3): Something received!!! from 6
0:0:48.914596529 DEBUG (3): posting receiveAggSumTask()!!!!
0:0:48.914596539 DEBUG (3): ReceiveAggSumTask():received msg...
0:0:48.914596539 DEBUG (3): ReceiveAggSumTask(): len=2
0:0:48.914596539 DEBUG (3): receiveAggSumTask(): sum= 35
0:0:48.914596539 DEBUG (3): Inside the ReceiveAggSumTask(): New agg_sum = 81
0:0:48.914764375 DEBUG (6): Inside the AggSumAMSend.sendDone()
0:0:48.914764375 DEBUG (6): A AggregationSum package sent... True
0:0:48.917770379 DEBUG (1): ### AggSumReceive.receive() start #####
0:0:48.917770379 DEBUG (1): Something received!!! from 7
0:0:48.917770379 DEBUG (1): posting receiveAggSumTask()!!!!
0:0:48.917770389 DEBUG (1): ReceiveAggSumTask():received msg...
0:0:48.917770389 DEBUG (1): ReceiveAggSumTask(): len=2
0:0:48.917770389 DEBUG (1): receiveAggSumTask(): sum= 59
0:0:48.917770389 DEBUG (4): Inside the ReceiveAggSumTask(): New agg_sum = 105
```

Και ομίως για την τριτη εποχη θα σας κανω μονο το σχημα για να μην γεμισει η αναφορα με φωτογραφιες.

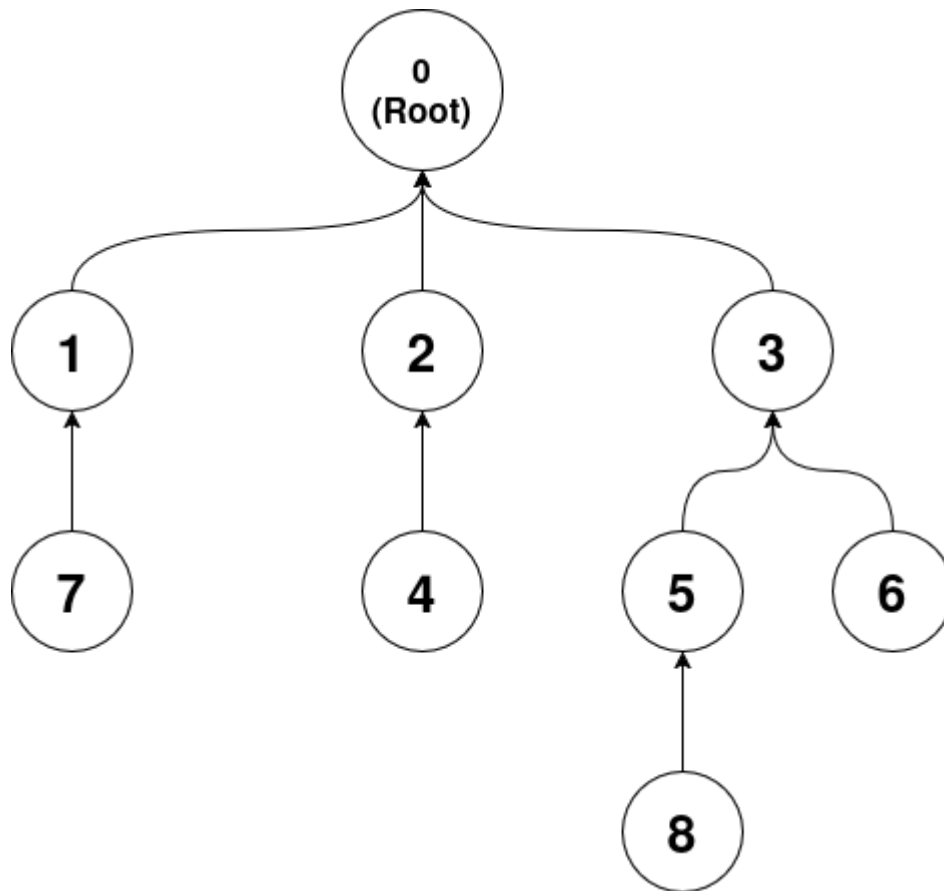


Και ειμαστε τυχερη γιατι σε αυτην την εποχη σταλθηκαν ολα τα μηνυματα τελεια.

Query: Avg

Εδώ έχουμε την προσομοίωση για το Avg και μας εβγαλε αυτο το δεντρο

```
0:0:10.199462896 DEBUG (3): NodeID 3: parentID= 0 , curdepth= 1 , aggType= 3  
0:0:10.199462896 DEBUG (1): NodeID 1: parentID= 0 , curdepth= 1 , aggType= 3  
0:0:10.199462896 DEBUG (2): NodeID 2: parentID= 0 , curdepth= 1 , aggType= 3  
0:0:10.397216811 DEBUG (6): NodeID 6: parentID= 3 , curdepth= 2 , aggType= 3  
0:0:10.397216811 DEBUG (5): NodeID 5: parentID= 3 , curdepth= 2 , aggType= 3  
0:0:10.399520875 DEBUG (4): NodeID 4: parentID= 2 , curdepth= 2 , aggType= 3  
0:0:10.402557356 DEBUG (7): NodeID 7: parentID= 1 , curdepth= 2 , aggType= 3  
0:0:10.593185447 DEBUG (8): NodeID 8: parentID= 5 , curdepth= 3 , aggType= 3
```

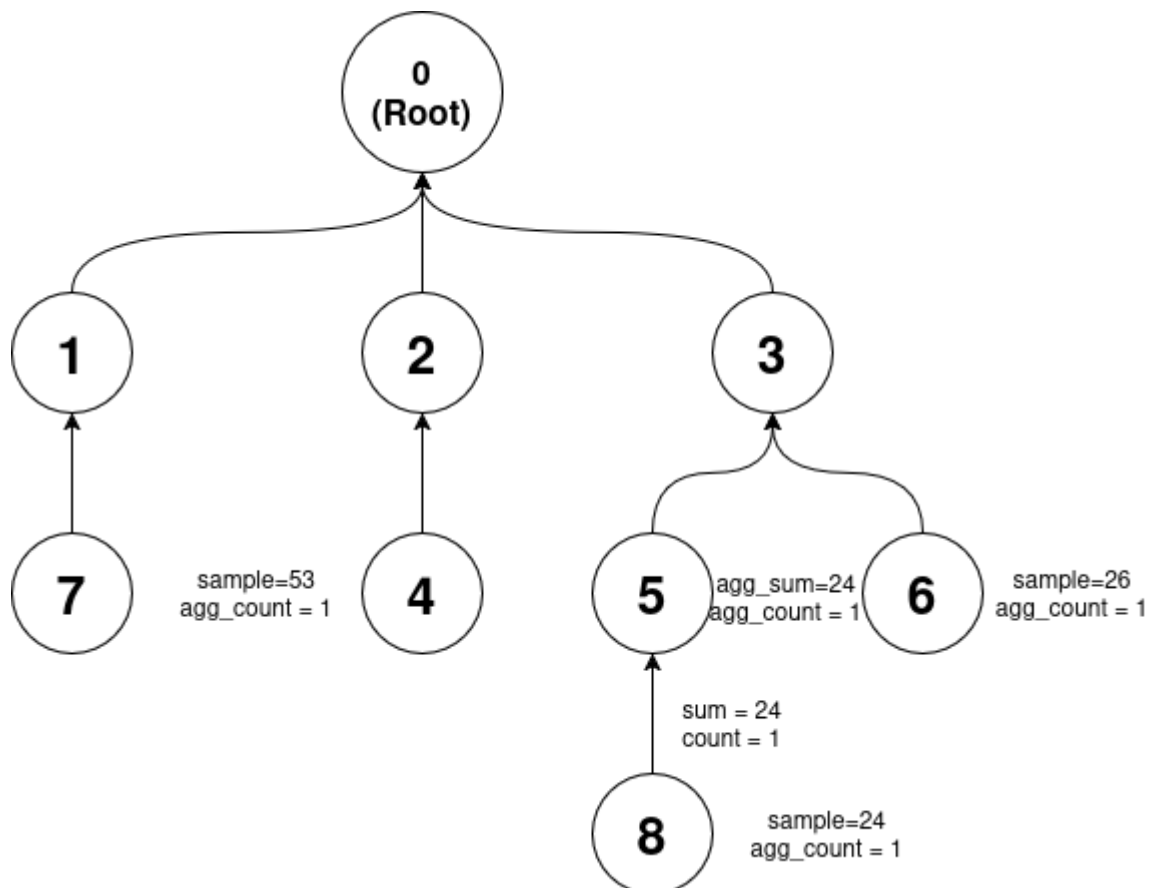


Αρχίζει ο κομβος 8 και παίρνει την πρώτη μετρηση που είναι 24, αυξανει το counter και στέλνει το μήναμα στον 5 ο οποίος το κάνει receive το αποθηκεύει ενώ ο 6 περνει την μετρηση του 26 και την στέλνει ομοιος και το 4 το 53

```

0:0:29.305664073 DEBUG (8): NodeID= 8 : AggregationAvg sample= 24 , BeforeCount= 0, BeforeSum= 0
0:0:29.305664073 DEBUG (8): NodeID= 8 : AggregationAvg CountAfter = 1, SumAfter= 24
0:0:29.305664073 DEBUG (8): NodeID= 8 : AggregationAvg value= 24/1
0:0:29.305664073 DEBUG (8): SendAggAvgTask() posted!!
0:0:29.305664073 DEBUG (8): AggregationAvg enqueued successfully in SendingQueue!!!
0:0:29.305664083 DEBUG (8): SendAggAvgTask(): Starting....
0:0:29.305664083 DEBUG (8): SendAggAvgTask(): Send returned success!!!
0:0:29.309646604 DEBUG (5): ### AggAvgReceive.receive() start #####
0:0:29.309646604 DEBUG (5): Something received!!! from 8
0:0:29.309646604 DEBUG (5): posting receiveAggAvgTask()!!!!
0:0:29.309646614 DEBUG (5): ReceiveAggAvgTask():received msg...
0:0:29.309646614 DEBUG (5): ReceiveAggAvgTask(): len=3
0:0:29.309646614 DEBUG (5): receiveAggAvgTask(): sum= 24, count = 1
0:0:29.309646614 DEBUG (5): Inside the ReceiveAggAvgTask(): New agg_sum = 24, New agg_count = 1
0:0:29.309814449 DEBUG (8): Inside the AggAvgAMSend.sendDone()
0:0:29.309814449 DEBUG (8): A AggregationAvg package sent... True
0:0:29.375976573 DEBUG (6): NodeID= 6 : AggregationAvg sample= 26 , BeforeCount= 0, BeforeSum= 0
0:0:29.375976573 DEBUG (6): NodeID= 6 : AggregationAvg CountAfter = 1, SumAfter= 26
0:0:29.375976573 DEBUG (6): NodeID= 6 : AggregationAvg value= 26/1
0:0:29.375976573 DEBUG (6): SendAggAvgTask() posted!!
0:0:29.375976573 DEBUG (6): AggregationAvg enqueued successfully in SendingQueue!!!
0:0:29.375976583 DEBUG (6): SendAggAvgTask(): Starting....
0:0:29.375976583 DEBUG (6): SendAggAvgTask(): Send returned success!!!
0:0:29.376953135 DEBUG (4): NodeID= 4 : AggregationAvg sample= 53 , BeforeCount= 0, BeforeSum= 0
0:0:29.376953135 DEBUG (4): NodeID= 4 : AggregationAvg CountAfter = 1, SumAfter= 53

```

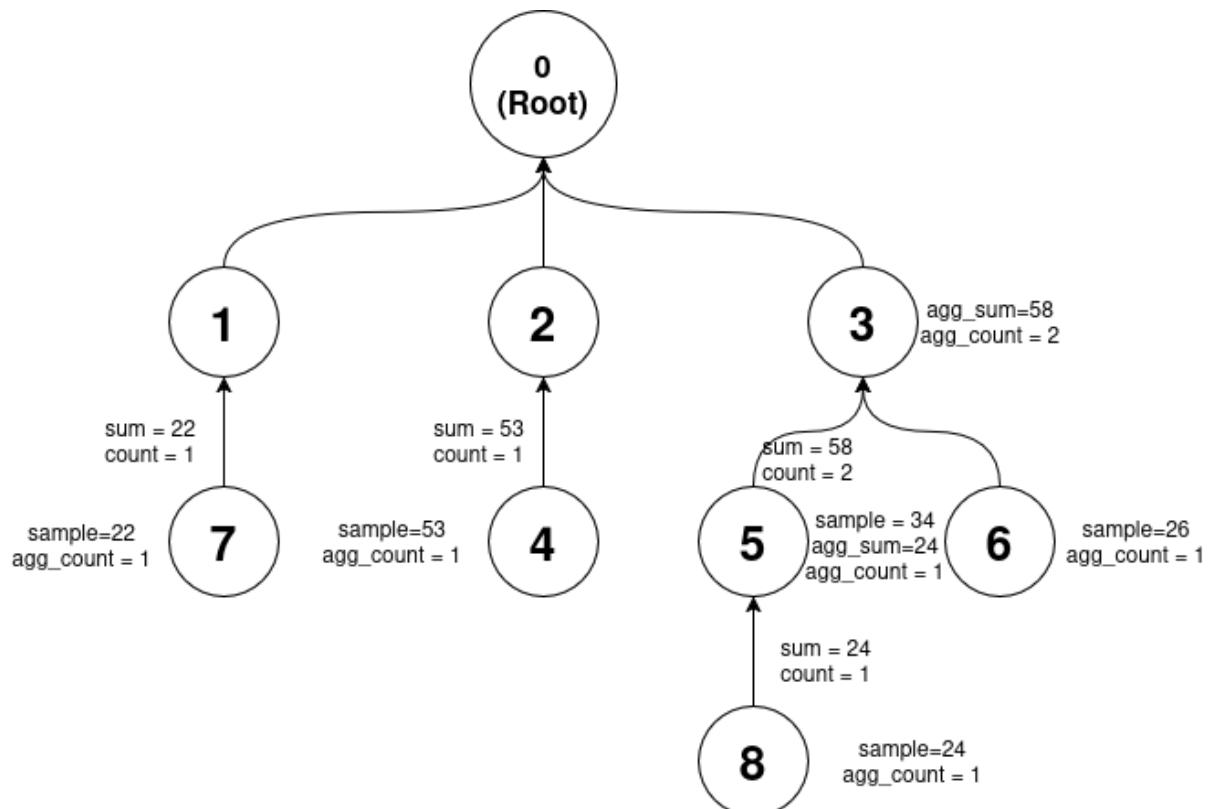


Μετα ο 7 βγαζει 22 και το στελνει ομοιος και ο 5 34 ειχε 24 που ειχε λαβει απο τον 8 αρα $34+24 = 58$ και το count 2. ο 3 λαμβανει το μηνυμα απο τον 5

```

0:0:29.376953135 DEBUG (4): SendAggAvgTask() posted!!
0:0:29.376953135 DEBUG (4): AggregationAvg enqueued successfully in SendingQueue!!!
0:0:29.376953145 DEBUG (4): SendAggAvgTask(): Starting....
0:0:29.376953145 DEBUG (4): SendAggAvgTask(): Send returned success!!!
0:0:29.377929698 DEBUG (7): NodeID= 7 : AggregationAvg sample= 22 , BeforeCount= 0, BeforeSum= 0
0:0:29.377929698 DEBUG (7): NodeID= 7 : AggregationAvg CountAfter = 1, SumAfter= 22
0:0:29.377929698 DEBUG (7): NodeID= 7 : AggregationAvg value= 22/1
0:0:29.377929698 DEBUG (7): SendAggAvgTask() posted!!
0:0:29.377929698 DEBUG (7): AggregationAvg enqueued successfully in SendingQueue!!!
0:0:29.377929708 DEBUG (7): SendAggAvgTask(): Starting....
0:0:29.377929708 DEBUG (7): SendAggAvgTask(): Send returned success!!!
0:0:29.378906260 DEBUG (5): NodeID= 5 : AggregationAvg sample= 34 , BeforeCount= 1, BeforeSum= 24
0:0:29.378906260 DEBUG (5): NodeID= 5 : AggregationAvg CountAfter = 2, SumAfter= 58
0:0:29.378906260 DEBUG (5): NodeID= 5 : AggregationAvg value= 58/2
0:0:29.378906260 DEBUG (5): SendAggAvgTask() posted!!
0:0:29.378906260 DEBUG (5): AggregationAvg enqueued successfully in SendingQueue!!!
0:0:29.378906270 DEBUG (5): SendAggAvgTask(): Starting....
0:0:29.378906270 DEBUG (5): SendAggAvgTask(): Send returned success!!!
0:0:29.380935677 DEBUG (3): ### AggAvgReceive.receive() start #####
0:0:29.380935677 DEBUG (3): Something received!!! from 5
0:0:29.380935677 DEBUG (3): posting receiveAggAvgTask()!!!!
0:0:29.380935687 DEBUG (3): ReceiveAggAvgTask():received msg...
0:0:29.380935687 DEBUG (3): ReceiveAggAvgTask(): len=3
0:0:29.380935687 DEBUG (3): receiveAqgAvgTask(): sum= 58, count = 2

```

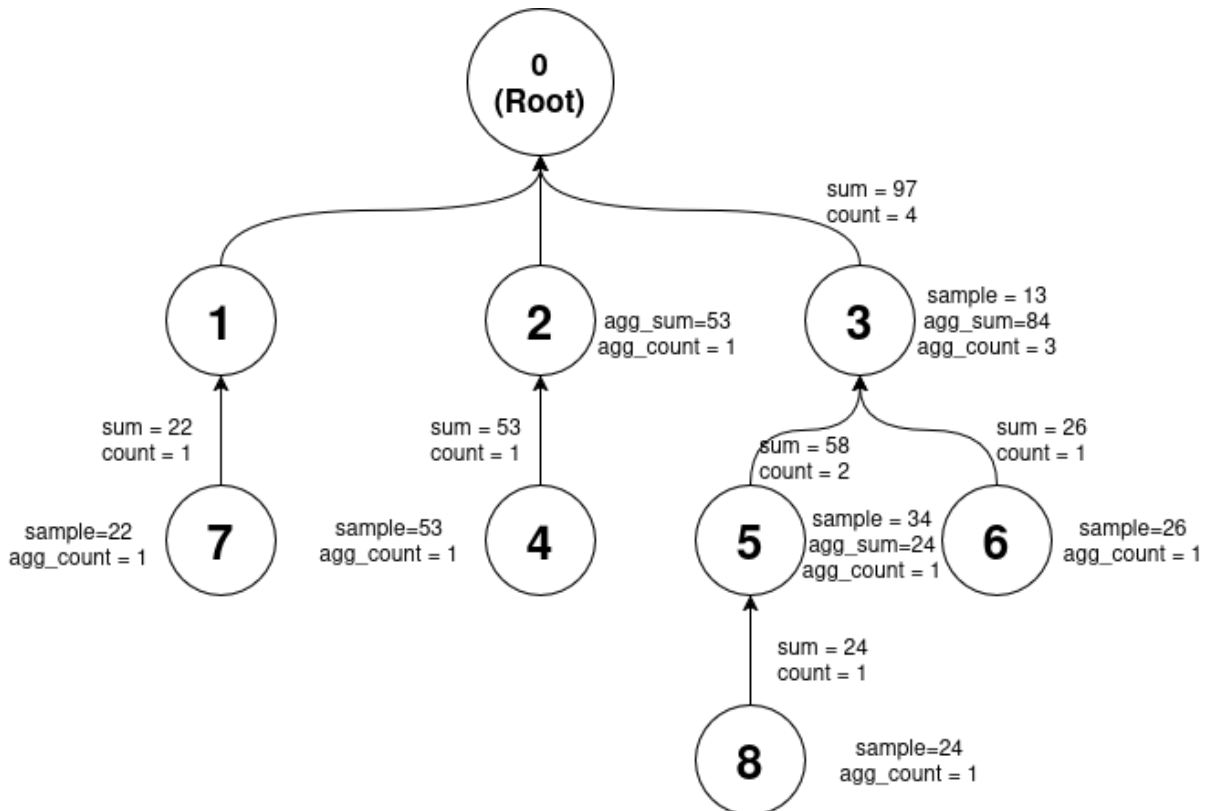


Επειτα ο 3 λαμβανει το μηνυμα του 6 και το προσθετει σε αυτα που ειχε και ο 2 λαμβανει το μηνυμα του 4. Ο 3 περνει την μετρηση του την προσθετη σε αυτα που ειχε και στελνει το μηνυμα

```

0:0:29.380935687 DEBUG (3): receiveAggAvgTask(). sum= 58, count = 2
0:0:29.380935687 DEBUG (3): Inside the ReceiveAggAvgTask(): New agg_sum = 58, New agg_count = 2
0:0:29.381103523 DEBUG (5): Inside the AggAvgAMSend.sendDone()
0:0:29.381103523 DEBUG (5): A AggregationAvg package sent... True
0:0:29.383087137 DEBUG (3): ### AggAvgReceive.receive() start #####
0:0:29.383087137 DEBUG (3): Something received!!! from 6
0:0:29.383087137 DEBUG (3): posting receiveAggAvgTask()!!!!
0:0:29.383087147 DEBUG (3): ReceiveAggAvgTask():received msg...
0:0:29.383087147 DEBUG (3): ReceiveAggAvgTask(): len=3
0:0:29.383087147 DEBUG (3): receiveAggAvgTask(): sum= 26, count = 1
0:0:29.383087147 DEBUG (3): Inside the ReceiveAggAvgTask(): New agg_sum = 84, New agg_count = 3
0:0:29.383102401 DEBUG (2): ### AggAvgReceive.receive() start #####
0:0:29.383102401 DEBUG (2): Something received!!! from 4
0:0:29.383102401 DEBUG (2): posting receiveAggAvgTask()!!!!
0:0:29.383102411 DEBUG (2): ReceiveAggAvgTask():received msg...
0:0:29.383102411 DEBUG (2): ReceiveAggAvgTask(): len=3
0:0:29.383102411 DEBUG (2): receiveAggAvgTask(): sum= 53, count = 1
0:0:29.383102411 DEBUG (2): Inside the ReceiveAggAvgTask(): New agg_sum = 53, New agg_count = 1
0:0:29.383254983 DEBUG (6): Inside the AggAvgAMSend.sendDone()
0:0:29.383254983 DEBUG (6): A AggregationAvg package sent... True
0:0:29.383270247 DEBUG (4): Inside the AggAvgAMSend.sendDone()
0:0:29.383270247 DEBUG (4): A AggregationAvg package sent... True
0:0:29.385238625 DEBUG (7): Inside the AggAvgAMSend.sendDone()
0:0:29.385238625 DEBUG (7): A AggregationAvg package sent... True
0:0:29.453125010 DEBUG (3): NodeID= 3 : AggregationAvg sample= 13 , BeforeCount= 3, BeforeSum=
84

```

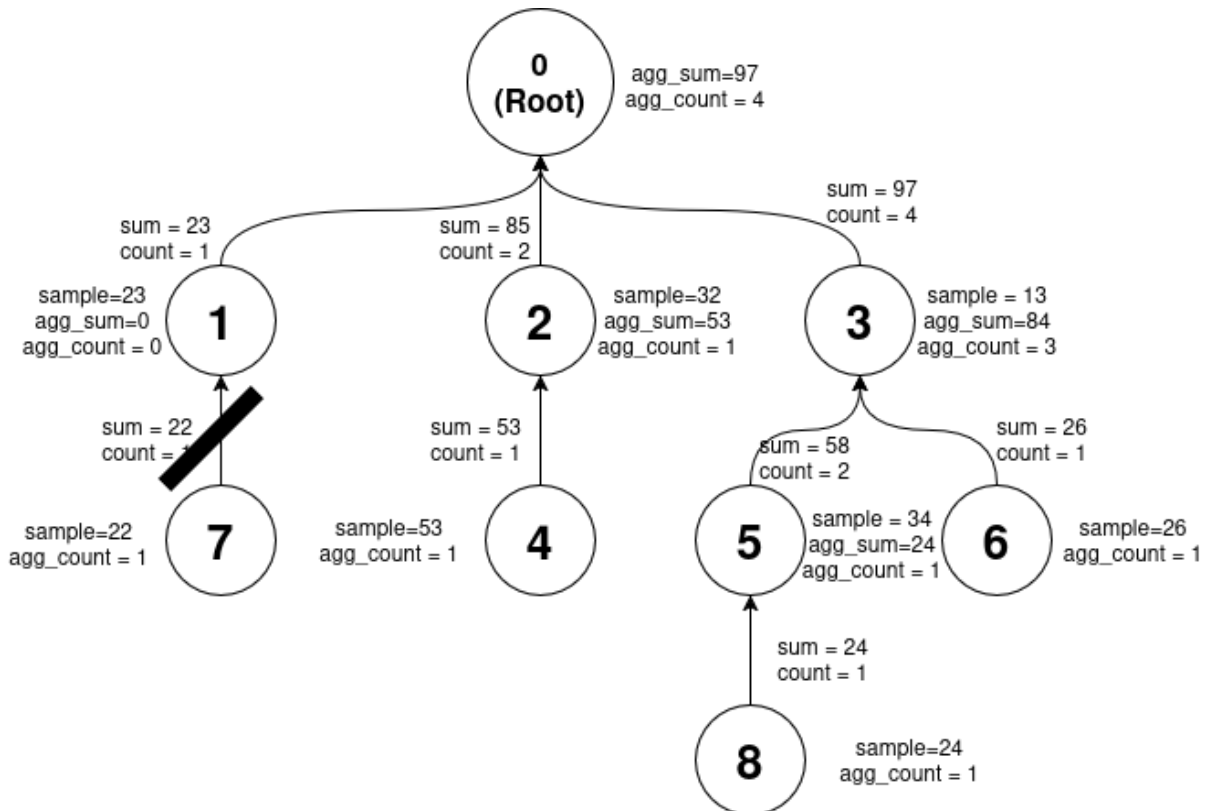


Ο root λαμβάνει από τον 3 το μήνυμα ενώ ο 2 περνεί την μετρηση 32 και στέλνει και ομοίως και ο 1 την μετρηση 23 και την στέλνει. Όπως βλέπουμε ο 1 δεν έλαβε ποτέ το μήνυμα από τον 7

```

0:0:29.453125010 DEBUG (3): NodeID= 3 : AggregationAvg CountAfter = 4, SumAfter= 97
0:0:29.453125010 DEBUG (3): NodeID= 3 : AggregationAvg value= 97/4
0:0:29.453125010 DEBUG (3): SendAggAvgTask() posted!!
0:0:29.453125010 DEBUG (3): AggregationAvg enqueued successfully in SendingQueue!!!
0:0:29.453125020 DEBUG (3): SendAggAvgTask(): Starting....
0:0:29.453125020 DEBUG (3): SendAggAvgTask(): Send returned success!!!
0:0:29.458648669 DEBUG (0): ### AggAvgReceive.receive() start #####
0:0:29.458648669 DEBUG (0): Something received!!! from 3
0:0:29.458648669 DEBUG (0): posting receiveAggAvgTask()!!!!
0:0:29.458648679 DEBUG (0): ReceiveAggAvgTask():received msg...
0:0:29.458648679 DEBUG (0): ReceiveAggAvgTask(): len=3
0:0:29.458648679 DEBUG (0): receiveAggAvgTask(): sum= 97, count = 4
0:0:29.458648679 DEBUG (0): Inside the ReceiveAggAvgTask(): New agg_sum = 97, New agg_count = 4
0:0:29.458816515 DEBUG (3): Inside the AggAvgAMSend.sendDone()
0:0:29.458816515 DEBUG (3): A AggregationAvg package sent... True
0:0:29.460937510 DEBUG (2): NodeID= 2 : AggregationAvg sample= 32 , BeforeCount= 1, BeforeSum= 53
0:0:29.460937510 DEBUG (2): NodeID= 2 : AggregationAvg CountAfter = 2, SumAfter= 85
0:0:29.460937510 DEBUG (2): NodeID= 2 : AggregationAvg value= 85/2
0:0:29.460937510 DEBUG (2): SendAggAvgTask() posted!!
0:0:29.460937510 DEBUG (2): AggregationAvg enqueued successfully in SendingQueue!!!
0:0:29.460937520 DEBUG (2): SendAggAvgTask(): Starting....
0:0:29.460937520 DEBUG (2): SendAggAvgTask(): Send returned success!!!
0:0:29.461914072 DEBUG (1): NodeID= 1 : AggregationAvg sample= 23 , BeforeCount= 0, BeforeSum= 0
0:0:29.461914072 DEBUG (1): NodeID= 1 : AggregationAvg CountAfter = 1, SumAfter= 23
0:0:29.461914072 DEBUG (1): NodeID= 1 : AggregationAvg value= 23/1

```

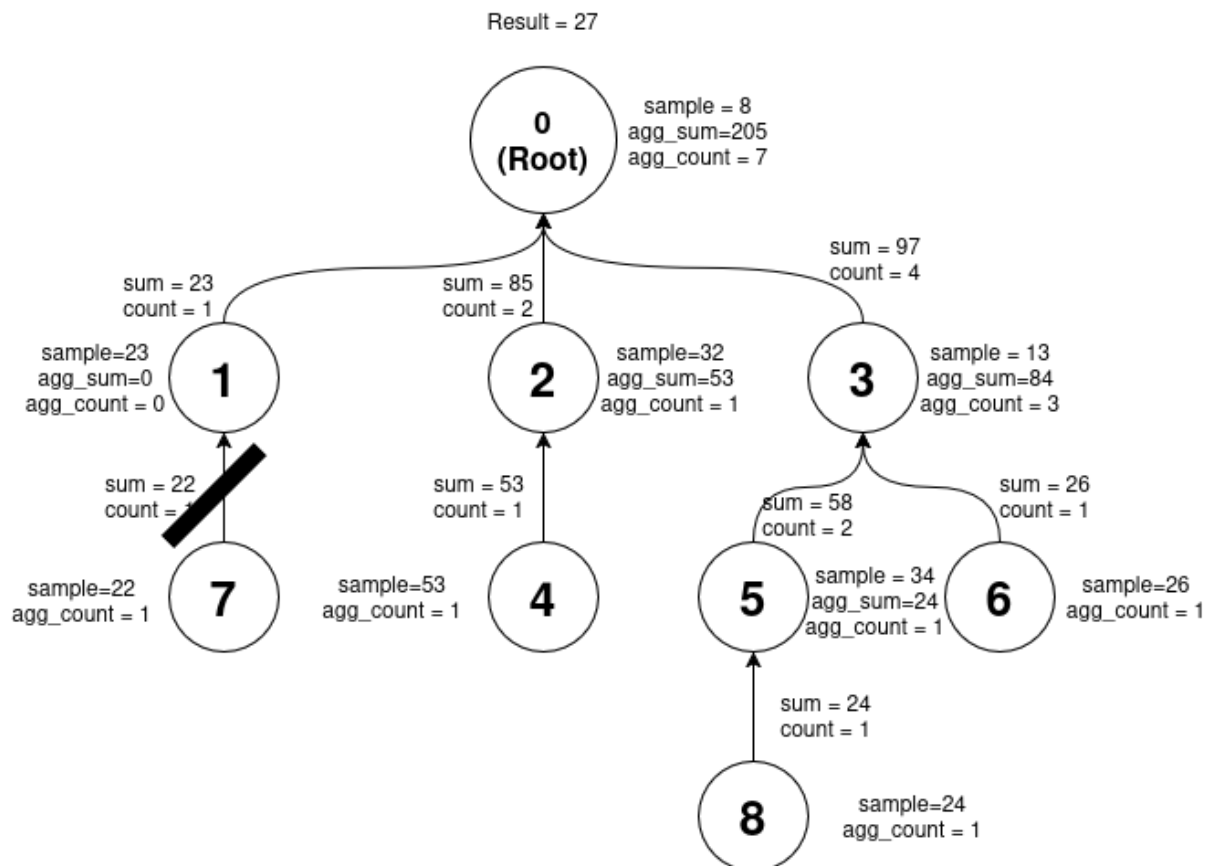


Τελος ο Root λαμβάνει τα μηνυματα απο τον 1 και τον 2 περνει την δικη του μετρηση που είναι 8 και τυπώνει το τελικο αποτελεσμα

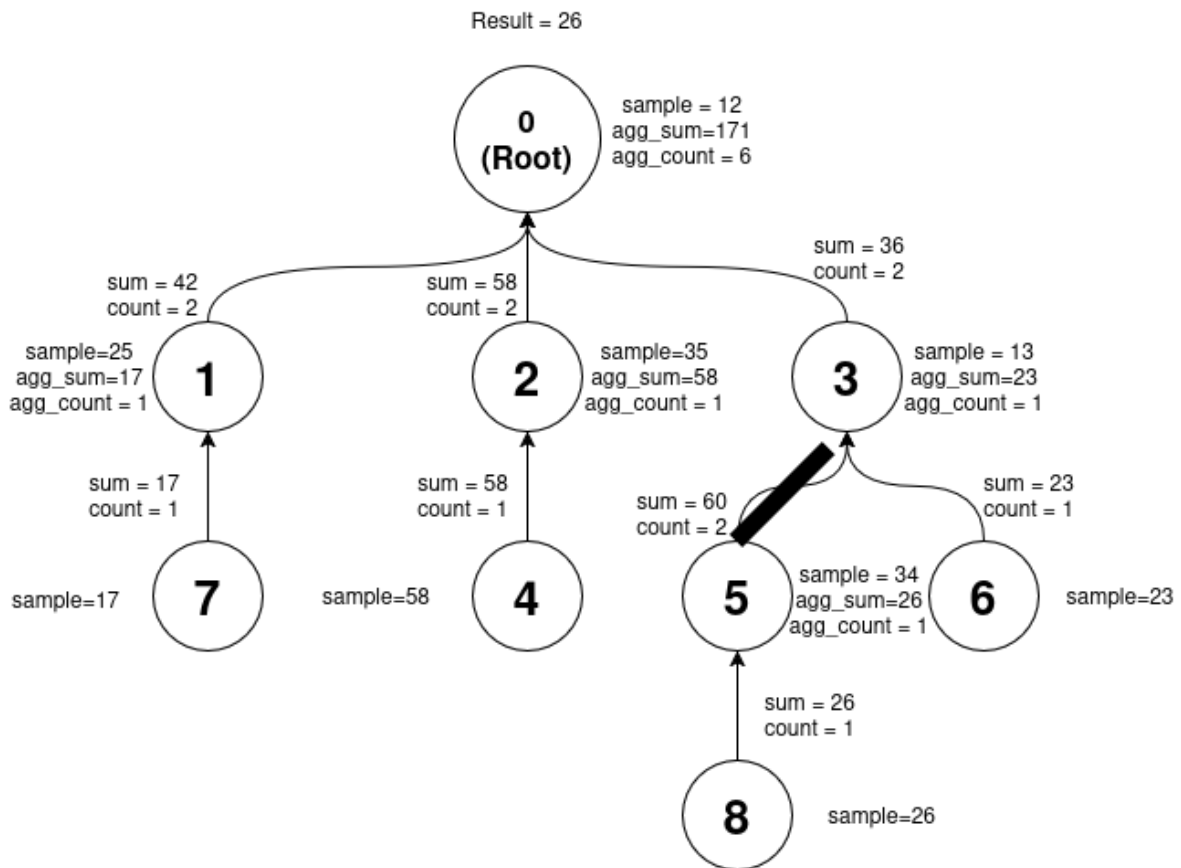
```

0:0:29.468460059 DEBUG (0): Something received!!! from 2
0:0:29.468460059 DEBUG (0): posting receiveAggAvgTask()!!!!
0:0:29.468460069 DEBUG (0): ReceiveAggAvgTask():received msg...
0:0:29.468460069 DEBUG (0): ReceiveAggAvgTask(): len=3
0:0:29.468460069 DEBUG (0): receiveAggAvgTask(): sum= 85, count = 2
0:0:29.468460069 DEBUG (0): Inside the ReceiveAggAvgTask(): New agg_sum = 182, New agg_count = 6
0:0:29.468627905 DEBUG (2): Inside the AggAvgAMSend.sendDone()
0:0:29.468627905 DEBUG (2): A AggregationAvg package sent... True
0:0:29.471511804 DEBUG (0): ### AggAvgReceive.receive() start #####
0:0:29.471511804 DEBUG (0): Something received!!! from 1
0:0:29.471511804 DEBUG (0): posting receiveAggAvgTask()!!!!
0:0:29.471511814 DEBUG (0): ReceiveAggAvgTask():received msg...
0:0:29.471511814 DEBUG (0): ReceiveAggAvgTask(): len=3
0:0:29.471511814 DEBUG (0): receiveAggAvgTask(): sum= 23, count = 1
0:0:29.471511814 DEBUG (0): Inside the ReceiveAggAvgTask(): New agg_sum = 205, New agg_count = 7
0:0:29.471679650 DEBUG (1): Inside the AggAvgAMSend.sendDone()
0:0:29.471679650 DEBUG (1): A AggregationAvg package sent... True
0:0:29.540039072 DEBUG (0): NodeID= 0 : AggregationAvg sample= 12 , BeforeCount= 7, BeforeSum= 205
0:0:29.540039072 DEBUG (0): NodeID= 0 : AggregationAvg CountAfter = 8, SumAfter= 217
0:0:29.540039072 DEBUG (0): ***////////////////\\***
0:0:29.540039072 DEBUG (0): AGG RESULT epoch=1 AVG=217/8=27
0:0:29.540039072 DEBUG (0): ***\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\***
0:0:48.826814072 DEBUG (8): NodeID= 8 : AggregationAvg sample= 26 , BeforeCount= 0, BeforeSum= 0

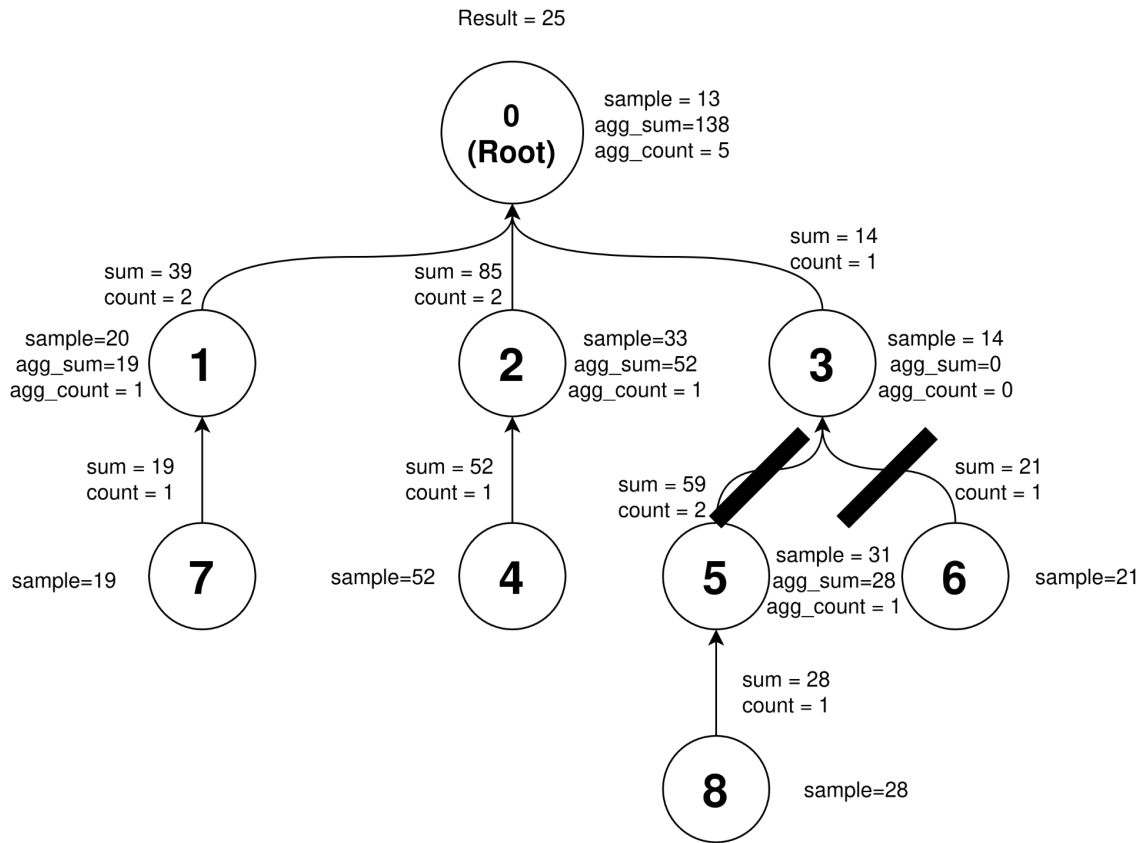
```



Για της επομενες δυο εποχες θα εχουμε μονο το σχημα. Το log της προσομοιωσεις βρισκεται
μαζι με τον κωδικα στο zip
epoch=2



epoch=3



Είρασαν ατυχοι και στις 3 εροch χαθηκαν μηνυματα. Παρολα αυτα βλεπουμε οτι δουλευει κανονικα.

5.Δουλεια του καθε φοιτητη

Την ασκηση την δουλεψαμε μαζι στο ιδιο χωρο. Οποτε δεν μπορουμε μπορουμε να πουμε ακριβες διαχωρισμο. Παρολα αυτα ο Αντωνης Μουτσοπουλος ασχοληθηκε λιγο παραπανω με το ξεσκαρταρισμα του κωδικα για να δουλευει σωστα σαν TAG ενω ο Κοσμας Σπυριδακης ασχοληθηκε πιο πολυ με την παρουσιαση.

6.Δουλεια για το δευτερο κομματι

Υπαρχει ενδεχομενο το random offset να κανει πιο πολυ κακο παρα καλο γιατι οπως ειπαμε δεν εχουμε ακριβη παραθυρα για να ακουει να επεξεργαζεται και να στελνει ο αισθητηρας, αλλα ενα γενικο. και με το offset χαλαει τον συγχρονισμο και μπορει να χαθουν καποια μηνυματα. Στην επομενη φαση θα κανουμε πιο πολυ testing και θα δουμε να ορισουμε ακριβη παραθυρα.